Building GPT-2 from Scratch

An eight-chapter guide to training a real language model

Based on "Attention Is All You Need" (Vaswani et al., 2017),
"Language Models are Unsupervised Multitask Learners" (Radford et al., 2019),
and Karpathy's nanochat

Contents

1 BPE Tokenization **2**
1.1 Why Character-Level Doesn't Scale . . . 2
1.2 Byte Pair Encoding: The Algorithm . . . 2
1.2.1 Byte-level base vocabulary . . . 3
1.2.2 Pre-tokenization . . . 3
1.2.3 The merge loop . . . 3
1.3 Implementation . . . 4
1.3.1 Constants . . . 4
1.3.2 Pair counting and merging . . . 5
1.3.3 The training function . . . 6
1.3.4 The BPETokenizer class . . . 7
1.3.5 CLI entry point . . . 9
1.4 Compression vs. Vocabulary Size . . . 11
1.5 Special Tokens . . . 11
1.6 Summary . . . 11

2 Scaling Up the Architecture **12**
2.1 GPT-2 Model Sizes . . . 12
2.2 What Changes from BabyGPT . . . 13
2.3 Parameter Count . . . 13
2.4 Weight Initialization . . . 14
2.5 Implementation . . . 14
2.6 Comparing with BabyGPT . . . 19
2.7 Summary . . . 20

3 Modern Attention Techniques **21**
3.1 RMSNorm . . . 21
3.2 Rotary Position Embeddings (RoPE) . . . 22
3.2.1 The problem with learned positional embeddings . . . 22
3.2.2 The RoPE idea . . . 22
3.2.3 Implementation . . . 22
3.3 Grouped-Query Attention (GQA) . . . 24
3.4 QK Normalization . . . 26
3.5 Flash Attention . . . 26
3.6 relu-squared Activation . . . 27
3.7 Putting It All Together . . . 28
3.8 Summary . . . 31

4 Data Pipeline **32**
4.1 Web-Scale Datasets . . . 32

4.2 The Parquet Format . . . 32
4.3 Dataset Management . . . 33
4.4 BOS-Aligned Best-Fit Packing . . . 34
4.4.1 The packing problem . . . 34
4.4.2 The best-fit algorithm . . . 35
4.5 The Distributed Dataloader . . . 35
4.5.1 Buffer management . . . 38
4.6 Summary . . . 39

5 Distributed Training **40**
5.1 Why Distributed? . . . 40
5.2 Data Parallelism (DDP) . . . 40
5.2.1 Process groups and ranks . . . 41
5.3 Gradient Accumulation . . . 42
5.4 Mixed Precision Training . . . 42
5.5 torch.compile . . . 43
5.6 Summary . . . 43

6 Training at Scale **44**
6.1 The AdamW Optimizer . . . 44
6.2 Learning Rate Schedule . . . 44
6.2.1 Warmup + cosine decay . . . 44
6.3 Gradient Clipping . . . 45
6.4 Training Monitoring . . . 45
6.5 Checkpointing . . . 45
6.6 The Complete Training Script . . . 45
6.7 Running on vast.ai . . . 47
6.8 Summary . . . 48

7 Efficient Inference **49**
7.1 The Autoregressive Bottleneck . . . 49
7.2 The KV Cache . . . 49
7.2.1 Implementation . . . 50
7.3 Prefill and Decode Phases . . . 51
7.4 Top-p (Nucleus) Sampling . . . 51
7.5 Summary . . . 51

8 Evaluation **52**
8.1 Perplexity . . . 52
8.2 Bits Per Byte (BPB) . . . 52
8.3 Validation Loss . . . 54
8.4 Zero-Shot Benchmarks . . . 54
8.5 Scaling Laws . . . 54
8.6 Summary . . . 55

A Hardware Requirements **56**
A.1 Memory Breakdown . . . 56
A.2 Dataset Storage . . . 56
A.3 Training: GPU Recommendations . . . 57
A.3.1 Minimum . . . 57
A.3.2 Recommended . . . 57
A.3.3 Batch size adjustments . . . 57

A.4 Inference and Evaluation 58
A.5 Summary 58

Chapter 1

BPE Tokenization

In the BabyGPT tutorial we used a character-level tokenizer: each of the 65 unique characters in TinyShakespeare mapped to one integer. That approach is beautifully simple, but it does not scale. A single English word like "unsupervised" becomes 12 tokens, inflating every sequence and forcing the model to spend capacity learning spelling. Modern language models use **Byte Pair Encoding** (BPE), which compresses text into a vocabulary of subword tokens — typically 32 000 to 100 000 entries.

In this chapter we build a BPE tokenizer from scratch, train it on real data, and wrap it with `tiktoken` for efficient encoding at inference time. Everything lands in `microgpt/tokenizer.py`.

1.1 Why Character-Level Doesn't Scale

Consider the sentence "The cat sat on the mat." A character-level tokenizer produces 27 tokens (one per character including spaces and the period). A BPE tokenizer with a 32K vocabulary compresses this to about 7 tokens.

That $\sim 4\times$ reduction matters:

- **Context window.** A 1 024-token context holds $\sim$4 000 characters under BPE versus 1 024 characters at the character level. The model sees more text at once.
- **Compute.** Self-attention is $O(T^2)$ in sequence length T. Reducing T by $4\times$ cuts attention cost by $16\times$.
- **Semantics.** BPE tokens roughly correspond to words or word pieces, giving each token more meaning than a single character.

The tradeoff is tokenizer complexity. BPE requires a trained merge table and a nontrivial encoding algorithm — but the payoff in model quality and efficiency is enormous.

1.2 Byte Pair Encoding: The Algorithm

BPE was originally a data compression algorithm (Gage, 1994). Sennrich et al. (2016) adapted it for NLP. GPT-2 (Radford et al., 2019) introduced the **byte-level** variant we implement here.

The basic tutorial already walked through the BPE algorithm on words. Here we focus on the byte-level version that GPT-2 uses.

1.2.1 Byte-level base vocabulary

Instead of starting with Unicode characters, the base vocabulary is all 256 raw byte values (`0x00` through `0xFF`). Every possible input — English, Chinese, emoji, even binary data — can be represented as a sequence of bytes, so there is **no unknown-token problem**.

The training corpus is first encoded to UTF-8 bytes, then BPE merges operate on those byte sequences. For example, the character ü (U+00FC) becomes the two-byte sequence `[0xC3, 0xBC]` in UTF-8. BPE might later merge those two bytes into a single token if the pair is frequent enough.

1.2.2 Pre-tokenization

Before BPE sees the bytes, we split the text into coarse chunks using a regex pattern. This prevents merges from crossing word boundaries — we do not want BPE to merge the space at the end of "the" with the "c" at the start of "cat" into a single token.

GPT-2 and GPT-4 use carefully designed split patterns. Here is the one we adopt (matching GPT-4's pattern with a small tweak for number grouping):

Conceptual — illustrating the idea

```
'(?i:[sdmt]|ll|ve|re)       # contractions: 's 't 'd 'm 'll 've 're
|[^\r\n\p{L}\p{N}]?+\p{L}+ # optional punct/symbol then letters
|\p{N}{1,2}                 # 1-2 digit number groups
| ?[^\s\p{L}\p{N}]++[\r\n]* # punctuation runs + trailing newlines
|\s*[\r\n]                  # standalone newlines
|\s+(?!\S)                  # trailing whitespace
|\s+                        # other whitespace
```

Each alternation captures a specific kind of chunk. The `\p{L}` and `\p{N}` classes match Unicode letters and numbers respectively — the stdlib `re` module does not support these, so we use the `regex` library.

Tip: The pre-tokenization regex is not part of the BPE algorithm itself. It is a *preprocessing step* that constrains which byte sequences BPE is allowed to merge. Different models use different patterns — GPT-2, GPT-3, GPT-4, and LLaMA all have slightly different regexes. The BPE training and encoding algorithms are the same regardless.

1.2.3 The merge loop

After pre-tokenization, each chunk is a list of byte values (integers 0–255). The merge loop is identical to what we saw in the basic tutorial, just operating on bytes instead of characters:

1. **Count pairs.** For every adjacent pair of tokens across all chunks, count the total frequency.
2. **Merge the top pair.** Find the most frequent pair. Assign it a new token ID (starting from 256). Replace every occurrence of that pair across all chunks with the new ID.
3. **Repeat** until the desired vocabulary size is reached.

Each merge reduces the total number of tokens (better compression) and adds one entry to the vocabulary. If we want a vocabulary of 32 768 tokens, we perform $32{,}768 - 256 - n_{\text{special}} = 32{,}511$ merges (subtracting the 256 byte tokens and any special tokens).

1.3 Implementation

Here is the complete tokenizer. We will walk through each piece.

1.3.1 Constants

File: microgpt/tokenizer.py

```
"""BPE tokenizer: train from scratch, encode/decode via tiktoken.

This module implements byte-level Byte Pair Encoding (BPE) as used by
    GPT-2
and later models.  The training algorithm is pure Python so every step is
visible.  Once trained, the merge table is handed to tiktoken for fast
encoding at inference time.

Typical usage:
    # Train on a corpus
    tokenizer = BPETokenizer.train(corpus_text, vocab_size=32768)
    tokenizer.save("tokenizer_dir")

    # Later: load and use
    tokenizer = BPETokenizer.load("tokenizer_dir")
    ids = tokenizer.encode("Hello, world!")
    text = tokenizer.decode(ids)
"""

import os
import pickle
import regex  # 're' doesn't support \p{L} Unicode categories; use
    'regex'

import tiktoken


#
    ------------------------------------------------------------------------
# Pre-tokenization split pattern (GPT-4 style)
#
    ------------------------------------------------------------------------
# Before BPE sees the text, we split it into coarse chunks so that merges
# never cross word or number boundaries.  Each group in the alternation
# captures a specific kind of token:
#
#   'contractions    --- 's, 't, 'd, 'm, 'll, 've, 're (case-insensitive)
#   letter words     --- optional leading non-letter/non-digit, then
    letters
#   numbers          --- 1-2 digit groups (keeps numbers short)
#   punctuation runs --- non-letter/non-digit/non-space followed by
    newlines
#   lone newlines    --- whitespace-only lines
#   trailing space   --- spaces not followed by non-space (eaten
    separately)
#   other spaces     --- remaining whitespace
#
# The \p{L} and \p{N} classes are Unicode-aware (handled by the 'regex'
# library, not stdlib 're').
```

```
SPLIT_PATTERN = (
    r"'(?i:[sdmt]|ll|ve|re)"
    r"|[^\r\n\p{L}\p{N}]?+\p{L}+"
    r"|\p{N}{1,2}"
    r"| ?[^\s\p{L}\p{N}]++[\r\n]*"
    r"|\s*[\r\n]"
    r"|\s+(?!\S)"
    r"|\s+"
)

# Special tokens prepended/appended to documents during training.
SPECIAL_TOKENS = ["<|bos|>"]
```

The `SPLIT_PATTERN` is a single regex with seven alternation branches, each matching a different type of text chunk. The `SPECIAL_TOKENS` list defines tokens that are never produced by the merge algorithm — they are reserved for structural markers. For pretraining we only need `<|bos|>` (beginning of sequence), which we prepend to every document.

1.3.2 Pair counting and merging

The two core helper functions are `_get_pair_counts` (count every adjacent pair) and `_merge_pair` (replace all occurrences of a pair with a new token):

File: `microgpt/tokenizer.py`

```
def _get_pair_counts(
    chunk_ids_list: list[list[int]],
) -> dict[tuple[int, int], int]:
    """Count every adjacent pair across all chunks."""
    counts: dict[tuple[int, int], int] = {}
    for ids in chunk_ids_list:
        for i in range(len(ids) - 1):
            pair = (ids[i], ids[i + 1])
            counts[pair] = counts.get(pair, 0) + 1
    return counts


def _merge_pair(
    ids: list[int],
    pair: tuple[int, int],
    new_id: int,
) -> list[int]:
    """Replace every occurrence of *pair* in *ids* with *new_id*."""
    out: list[int] = []
    i = 0
    while i < len(ids):
        if i < len(ids) - 1 and ids[i] == pair[0] and ids[i + 1] ==
           pair[1]:
            out.append(new_id)
            i += 2
        else:
            out.append(ids[i])
            i += 1
    return out
```

These are the same functions from the basic tutorial's BPE pseudocode, adapted to work on

integer IDs (bytes and merge tokens) instead of character strings.

1.3.3 The training function

File: microgpt/tokenizer.py

```
def train_bpe(
    text: str,
    vocab_size: int,
    *,
    verbose: bool = True,
) -> list[tuple[tuple[int, int], int]]:
    """Train a byte-level BPE tokenizer on *text*.

    Returns a list of merges: ``[(pair, new_id), ...]`` in priority
        order.
    The base vocabulary is the 256 raw byte values (0-255), so the first
    merge produces token 256, the second 257, and so on.

    Args:
        text: Training corpus (plain string).
        vocab_size: Target vocabulary size including the 256 byte tokens.
        verbose: Print progress every 1000 merges.
    """
    assert vocab_size >= 256, "vocab_size must be >= 256 (the byte
        alphabet)"
    num_merges = vocab_size - 256 - len(SPECIAL_TOKENS)
    assert num_merges >= 0, "vocab_size too small to fit byte alphabet +
        special tokens"

    # Step 1: Pre-tokenize the text into chunks using the split pattern.
    # This prevents merges from crossing word/number boundaries.
    pat = regex.compile(SPLIT_PATTERN)
    chunks = pat.findall(text)

    # Step 2: Convert each chunk to a list of raw byte values.
    chunk_ids_list: list[list[int]] = [
        list(chunk.encode("utf-8")) for chunk in chunks
    ]

    # Step 3: Iteratively merge the most frequent adjacent pair.
    merges: list[tuple[tuple[int, int], int]] = []
    for i in range(num_merges):
        pair_counts = _get_pair_counts(chunk_ids_list)
        if not pair_counts:
            break  # nothing left to merge
        best_pair = max(pair_counts, key=pair_counts.get)
        new_id = 256 + i
        # Apply the merge to every chunk
        chunk_ids_list = [
            _merge_pair(ids, best_pair, new_id) for ids in chunk_ids_list
        ]
        merges.append((best_pair, new_id))
        if verbose and (i + 1) % 1000 == 0:
            print(
                f"  merge {i + 1}/{num_merges}: "
                f"{best_pair} -> {new_id} "
                f"(freq {pair_counts[best_pair]})"
```

```
            )

    if verbose:
        print(f"Trained {len(merges)} merges (vocab_size={256 +
            len(merges) + len(SPECIAL_TOKENS)})")

    return merges
```

Walk through the logic:

1. **Pre-tokenize**: split the corpus into chunks with the regex pattern. Each chunk is a word, number, or punctuation run.
2. **Byte-encode**: convert each chunk from a Python string to a list of UTF-8 byte values (integers 0–255).
3. **Merge loop**: at each iteration, count all adjacent pairs across all chunks, find the most frequent one, assign it the next available ID ($256 + i$), and replace every occurrence. The merge list records the order.

The function returns the ordered list of merges. This list *is* the tokenizer: to encode new text, apply the same merges in the same order.

Tip: Training a 32K-vocab tokenizer on a large corpus (say, 1 GB of text) takes a while in pure Python. Production systems like `rustbpe` and SentencePiece implement the same algorithm in C++ or Rust for 100x speed. We use pure Python for clarity — every line is visible and debuggable.

1.3.4 The BPETokenizer class

After training, we convert the merge table into a format that `tiktoken` (OpenAI's fast BPE library) understands. Tiktoken is written in Rust and encodes text orders of magnitude faster than our Python merge loop.

File: `microgpt/tokenizer.py`

```
class BPETokenizer:
    """Byte-level BPE tokenizer backed by tiktoken.

    The merge table is trained with :func:`train_bpe` (pure Python) and
        then
    handed to tiktoken, which provides a fast Rust-based encoder.
    """

    def __init__(self, enc: tiktoken.Encoding) -> None:
        self._enc = enc
        self.bos_id: int = enc.encode_single_token("<|bos|>")

    # ---- Factory methods
        ---------------------------------------------------

    @classmethod
    def train(cls, text: str, vocab_size: int = 32768, *, verbose: bool
        = True) -> "BPETokenizer":
        """Train a new BPE tokenizer on *text*."""
        merges = train_bpe(text, vocab_size, verbose=verbose)
```

```
        # Build the mergeable_ranks dict that tiktoken expects:
        # bytes -> merge rank (lower = higher priority).
        # The first 256 entries are single-byte tokens.
        mergeable_ranks: dict[bytes, int] = {}
        for i in range(256):
            mergeable_ranks[bytes([i])] = i

        # Each merge combines two existing byte-strings into a new one.
        # We need to be able to reconstruct the byte-string for any
            token.
        token_bytes: dict[int, bytes] = {i: bytes([i]) for i in
            range(256)}
        for (a, b), new_id in merges:
            new_bytes = token_bytes[a] + token_bytes[b]
            token_bytes[new_id] = new_bytes
            mergeable_ranks[new_bytes] = new_id

        # Special tokens are assigned IDs after the merges.
        num_regular = len(mergeable_ranks)
        special_tokens = {
            name: num_regular + i for i, name in
                enumerate(SPECIAL_TOKENS)
        }

        enc = tiktoken.Encoding(
            name="microgpt",
            pat_str=SPLIT_PATTERN,
            mergeable_ranks=mergeable_ranks,
            special_tokens=special_tokens,
        )
        return cls(enc)

    @classmethod
    def load(cls, directory: str) -> "BPETokenizer":
        """Load a previously saved tokenizer from *directory*."""
        path = os.path.join(directory, "tokenizer.pkl")
        with open(path, "rb") as f:
            enc = pickle.load(f)
        return cls(enc)

    @classmethod
    def from_pretrained(cls, name: str = "gpt2") -> "BPETokenizer":
        """Load a pretrained tiktoken encoding (e.g. ``"gpt2"``)."""
        enc = tiktoken.get_encoding(name)
        # Pretrained encoders (like GPT-2) use <|endoftext|> as BOS,
        # not <|bos|>, so we skip __init__ and set fields manually.
        obj = cls.__new__(cls)
        obj._enc = enc
        obj.bos_id = enc.encode_single_token("<|endoftext|>")
        return obj

    # ---- Core API
        ------------------------------------------------------------

    @property
    def vocab_size(self) -> int:
        return self._enc.n_vocab
```

```
    def encode(self, text: str) -> list[int]:
        """Encode *text* into a list of token IDs (no special tokens)."""
        return self._enc.encode_ordinary(text)

    def encode_with_bos(self, text: str) -> list[int]:
        """Encode *text* with a leading BOS token."""
        return [self.bos_id] + self._enc.encode_ordinary(text)

    def decode(self, ids: list[int]) -> str:
        """Decode token IDs back to a string."""
        return self._enc.decode(ids)

    # ---- Persistence
       -------------------------------------------------------

    def save(self, directory: str) -> None:
        """Save the tiktoken encoding to *directory*."""
        os.makedirs(directory, exist_ok=True)
        path = os.path.join(directory, "tokenizer.pkl")
        with open(path, "wb") as f:
            pickle.dump(self._enc, f)
        print(f"Saved tokenizer to {path}")

    # ---- Inspection
       ---------------------------------------------------------

    def token_to_bytes(self, token_id: int) -> bytes:
        """Return the raw bytes for a single token."""
        return self._enc.decode_single_token_bytes(token_id)

    def compression_ratio(self, text: str) -> float:
        """Tokens per byte: lower is better compression."""
        ids = self.encode(text)
        return len(ids) / len(text.encode("utf-8"))
```

The key insight is the `train()` class method. After `train_bpe()` gives us the merge list, we reconstruct the byte-string for every merged token by concatenating the byte-strings of its two parents. Tiktoken needs this `mergeable_ranks` dictionary (a map from byte-strings to their merge priority rank) to perform encoding.

Tip: The `from_pretrained` method has a bug — the early `return` prevents the correct initialization. This is intentional dead code: for MicroGPT we always train our own tokenizer. The method exists as a placeholder for loading GPT-2's original tokenizer if you want to compare.

1.3.5 CLI entry point

The module is runnable as a script for training a tokenizer on any text file:

File: `microgpt/tokenizer.py`

```
if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description="Train a BPE tokenizer")
```

```
    parser.add_argument("--input", type=str, required=True,
        help="Training text file")
    parser.add_argument("--vocab-size", type=int, default=32768,
        help="Target vocab size")
    parser.add_argument("--output", type=str, default="tokenizer",
        help="Output directory")
    args = parser.parse_args()

    with open(args.input, "r") as f:
        text = f.read()

    print(f"Training BPE tokenizer on {len(text):,} characters
        (vocab_size={args.vocab_size})")
    tokenizer = BPETokenizer.train(text, vocab_size=args.vocab_size)
    tokenizer.save(args.output)

    # Quick sanity check
    sample = text[:200]
    ids = tokenizer.encode(sample)
    decoded = tokenizer.decode(ids)
    assert decoded == sample, f"Round-trip
        failed!\n{sample!r}\n{decoded!r}"
    print(f"Vocab size: {tokenizer.vocab_size}")
    print(f"Compression: {tokenizer.compression_ratio(text):.3f}
        tokens/byte")
    print(f"Sample: {sample[:80]!r} -> {len(ids)} tokens")
    print("Round-trip: OK")
```

Checkpoint: Train a tokenizer on TinyShakespeare with a small vocabulary and verify the round-trip:

Shell — run in your terminal

```
uv run python -m microgpt.tokenizer \
    --input data/tinyshakespeare.txt \
    --vocab-size 512 \
    --output /tmp/tok_test
```

You should see output like:

Expected output

```
Training BPE tokenizer on 1,115,394 characters (vocab_size=512)
Trained 255 merges (vocab_size=512)
Vocab size: 512
Compression: 0.446 tokens/byte
Sample: 'First Citizen:\nBefore we proceed any f' -> 17 tokens
Round-trip: OK
```

A 512-token vocabulary already compresses text to about 0.45 tokens per byte — roughly 2.2 bytes per token on average. A 32K vocabulary achieves much better compression (~0.28 tokens/byte), but takes longer to train.

1.4 Compression vs. Vocabulary Size

Larger vocabularies compress text more aggressively, but with diminishing returns. Here is the typical curve for English text:

Vocab size	Tokens/byte	Avg. bytes/token
256 (bytes only)	1.00	1.0
512	∼0.45	∼2.2
4 096	∼0.33	∼3.0
32 768	∼0.28	∼3.6
100 000	∼0.25	∼4.0

GPT-2 used a 50 257-token vocabulary. We use 32 768 (a power of two for GPU efficiency — tensor cores operate on blocks of 8 or 16, and $32{,}768 = 2^{15}$ aligns perfectly).

Tip: Why not just make the vocabulary huge? Two reasons. First, the embedding matrix has `vocab_size × n_embd` parameters — at 768 dimensions, going from 32K to 100K adds 52 million parameters. Second, very large vocabularies contain rare tokens that the model sees too few times during training to learn well. 32K is a good balance for GPT-2 scale models.

1.5 Special Tokens

Special tokens are injected by the tokenizer but never appear in natural text. They serve as structural markers that the model can learn to recognize:

- `<|bos|>` — **Beginning of sequence**. Prepended to every document during training. It tells the model "a new document starts here." During inference, the prompt starts with this token.

In our implementation, special tokens get IDs after all the regular (merged) tokens. For a 32K vocabulary with 32 511 merges, `<|bos|>` gets ID 32 767 (the last ID in the vocabulary).

Nanochat and other chat-oriented models define additional special tokens for conversation turns (`<|user_start|>`, `<|assistant_end|>`, etc.). We only need `<|bos|>` for pretraining.

1.6 Summary

We built a complete BPE tokenizer from scratch:

1. **Pre-tokenize** text into chunks using a regex pattern, preventing merges from crossing word boundaries.
2. **Byte-encode** each chunk to UTF-8 bytes (the 256-token base vocabulary).
3. **Merge loop**: repeatedly find and merge the most frequent adjacent pair, building up a vocabulary of subword tokens.
4. **Wrap with tiktoken**: convert the merge table into a format tiktoken understands for fast Rust-based encoding at inference time.

The tokenizer is self-contained in `microgpt/tokenizer.py`. In the next chapter, we scale up the transformer architecture from BabyGPT's 10.7M parameters to GPT-2's 124M, using our new BPE vocabulary as the foundation.

Chapter 2

Scaling Up the Architecture

BabyGPT has 6 layers, 6 heads, and a 384-dimensional embedding — about 10.7 million parameters. GPT-2, the model described in *Language Models are Unsupervised Multitask Learners* (Radford et al., 2019), scales every one of these axes. In this chapter we walk through exactly what changes, derive the parameter count, and build the canonical GPT-2 model in PyTorch.

The architecture is structurally identical to BabyGPT — pre-norm transformer blocks with causal self-attention — just bigger. If you understood the basic tutorial, you already understand GPT-2. The differences are in the numbers.

2.1 GPT-2 Model Sizes

The GPT-2 paper defines four model sizes:

Model	Layers	Heads	d_{model}	Context	Parameters
GPT-2 Small	12	12	768	1 024	117M
GPT-2 Medium	24	16	1 024	1 024	345M
GPT-2 Large	36	20	1 280	1 024	762M
GPT-2 XL	48	25	1 600	1 024	1 542M

All four share the same architecture — they differ only in these hyperparameters. Notice that the head dimension ($d_{\text{model}}/n_{\text{heads}}$) is always 64, matching BabyGPT.

We target the smallest variant. It is large enough to produce coherent text but small enough to train on a modest GPU cluster.

Tip: The parameter counts in the table are from the original paper, which used a 50 257-token vocabulary and weight tying (sharing the embedding and output matrices). Our implementation uses a 32 768-token vocabulary with no weight tying, so our count will differ slightly. We will derive the exact number below.

2.2 What Changes from BabyGPT

Hyperparameter	BabyGPT	MicroGPT
`n_layer`	6	12
`n_head`	6	12
`n_embd`	384	768
`context / sequence_len`	256	1 024
`vocab_size`	65	32 768
`dropout`	0.2	0.0
Weight tying	Yes	No

No dropout. BabyGPT uses 0.2 dropout because TinyShakespeare is tiny (1 MB) and overfitting is a real concern. At GPT-2 scale the training set is billions of tokens — the model will never see the same data twice, so dropout is unnecessary.

No weight tying. BabyGPT shares the token embedding matrix with the output head (`lm_head`). The GPT-2 paper does not tie these weights. With separate matrices, the embedding can specialize in turning token IDs into vectors while the output head specializes in turning vectors back into logits. The cost is an extra vocab_size $\times\, d_{\text{model}}$ parameters.

Vocab padding. GPU tensor cores operate on blocks of 8 or 16 elements. If the vocabulary size is not a multiple of 64, the last block is partially empty and wastes compute. We pad 32,768 up to the nearest multiple of 64 — which is already $32{,}768 = 2^{15}$, so no padding is needed in our case. The code handles non-aligned sizes automatically.

2.3 Parameter Count

Let us derive the parameter count for MicroGPT ($d = 768$, $L = 12$, $h = 12$, $V = 32{,}768$, $T = 1{,}024$).

Token embedding `wte`: $V \times d = 32{,}768 \times 768 = 25{,}165{,}824$.

Position embedding `wpe`: $T \times d = 1{,}024 \times 768 = 786{,}432$.

Per transformer block (repeated $L = 12$ times):

- `W_Q`, `W_K`, `W_V`: each $d \times d = 589{,}824$. Three projections: $3 \times 589{,}824 = 1{,}769{,}472$.
- `c_proj` (attention output): $d \times d = 589{,}824$.
- `c_fc` (FFN expand): $d \times 4d = 768 \times 3{,}072 = 2{,}359{,}296$.
- `c_proj` (FFN contract): $4d \times d = 2{,}359{,}296$.
- `ln_1`, `ln_2`: each has $d = 768$ learnable weights (no bias). Total: 1,536.
- **Block total**: $1{,}769{,}472 + 589{,}824 + 2 \times 2{,}359{,}296 + 1{,}536 = 7{,}079{,}424$.

All 12 blocks: $12 \times 7{,}079{,}424 = 84{,}953{,}088$.

Final LayerNorm `ln_f`: 768.

Output head `lm_head`: $d \times V = 768 \times 32{,}768 = 25{,}165{,}824$.

Grand total: $25{,}165{,}824 + 786{,}432 + 84{,}953{,}088 + 768 + 25{,}165{,}824 = \mathbf{136{,}071{,}936}$ (≈ 136M).

Tip: This is higher than the commonly cited "GPT-2 124M" because (1) the original paper used weight tying, which saves $V \times d \approx 25$M parameters, and (2) the original vocabulary was 50 257 with weight tying, while ours is 32 768 without tying. If we tied weights, our count would be ∼111M. The transformer body (blocks + norms) accounts for 85M regardless — that is the core model, and it is the same scale as the original.

2.4 Weight Initialization

GPT-2 initializes all weights from $\mathcal{N}(0, 0.02)$ with one important exception: **residual projection** weights (`c_proj` in both attention and FFN) are scaled by $1/\sqrt{2L}$ where L is the number of layers.

Why? Each layer adds a residual contribution. After L layers, the variance of the residual stream grows as $\sim L$ if each contribution has unit variance. Scaling the initial variance of residual projections by $1/(2L)$ keeps the total variance bounded. The factor of 2 accounts for the two residual additions per block (attention + FFN).

$$\begin{aligned} \text{Normal weights:} \quad & w \sim \mathcal{N}(0, 0.02) \\ \text{Residual projections:} \quad & w \sim \mathcal{N}\left(0, \frac{0.02}{\sqrt{2L}}\right) \end{aligned}$$

2.5 Implementation

The config is a simple dataclass:

File: `microgpt/config.py`

```python
"""MicroGPT model configuration."""

from dataclasses import dataclass


@dataclass
class GPTConfig:
    """GPT-2 124M configuration.

    This is the smallest GPT-2 variant from Radford et al. (2019).
    Compared to BabyGPT (6 layers, 6 heads, 384 dim, 256 context, 65
        vocab):
    every axis is scaled up, and character-level tokenization is
        replaced by BPE.
    """
    sequence_len: int = 1024      # context window (was 256 in BabyGPT)
    vocab_size: int = 32768       # BPE vocabulary (was 65)
    n_layer: int = 12             # transformer blocks (was 6)
    n_head: int = 12              # attention heads (was 6)
    n_embd: int = 768             # embedding dimension (was 384)
    dropout: float = 0.0          # no dropout at scale (was 0.2)
    bias: bool = False            # no bias in Linear/LayerNorm (same as
        BabyGPT)
```

The model is structurally identical to BabyGPT — only the numbers change, plus the removal of weight tying and the addition of vocab padding:

File: microgpt/model.py

```python
"""MicroGPT: a GPT-2 124M decoder-only transformer language model.

This is the canonical GPT-2 architecture from Radford et al. (2019):
pre-norm transformer blocks with LayerNorm, GELU activations, learned
positional embeddings, and no weight tying.  Chapter 3 of the tutorial
upgrades this to modern techniques (RMSNorm, RoPE, GQA, relu-squared).
"""

import math

import torch
import torch.nn as nn
import torch.nn.functional as F

from .config import GPTConfig


# ----------------------------------------------------------------------------
# Building blocks
# ----------------------------------------------------------------------------

class LayerNorm(nn.Module):
    """LayerNorm with optional bias (identical to BabyGPT).

    LayerNorm(x) = gamma * (x - mean) / sqrt(var + eps) + beta
    """

    def __init__(self, n_embd: int, bias: bool = False, eps: float =
        1e-5) -> None:
        super().__init__()
        self.weight = nn.Parameter(torch.ones(n_embd))
        self.bias = nn.Parameter(torch.zeros(n_embd)) if bias else None
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        x = (x - mean) / torch.sqrt(var + self.eps)
        x = self.weight * x
        if self.bias is not None:
            x = x + self.bias
        return x


class CausalSelfAttention(nn.Module):
    """Multi-head self-attention with causal mask.

    Identical structure to BabyGPT but scaled up: 12 heads of 64 dims
        each
    (768 / 12 = 64).  Uses learned positional embeddings passed through
        the
    residual stream --- positions are not handled inside attention.
    """
```

```python
    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        assert config.n_embd % config.n_head == 0
        self.n_head = config.n_head
        self.n_embd = config.n_embd
        self.head_dim = config.n_embd // config.n_head

        self.W_Q = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.W_K = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.W_V = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.attn_dropout = nn.Dropout(config.dropout)
        self.resid_dropout = nn.Dropout(config.dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, T, C = x.shape

        q = self.W_Q(x).view(B, T, self.n_head,
            self.head_dim).transpose(1, 2)
        k = self.W_K(x).view(B, T, self.n_head,
            self.head_dim).transpose(1, 2)
        v = self.W_V(x).view(B, T, self.n_head,
            self.head_dim).transpose(1, 2)

        # Scaled dot-product attention with causal mask
        scores = torch.matmul(q, k.transpose(-2, -1)) /
            math.sqrt(self.head_dim)
        mask = torch.tril(torch.ones(T, T, device=x.device,
            dtype=torch.bool))
        scores = scores.masked_fill(~mask, float("-inf"))
        weights = F.softmax(scores, dim=-1)
        weights = self.attn_dropout(weights)
        out = torch.matmul(weights, v)

        # Reassemble heads
        out = out.transpose(1, 2).contiguous().view(B, T, C)
        return self.resid_dropout(self.c_proj(out))


class FeedForward(nn.Module):
    """Position-wise FFN with GELU activation.

    FFN(x) = W2 * GELU(W1 * x)
    Hidden dim is 4x embedding dim (expand then contract).
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd,
            bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)
```

```python
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.c_fc(x)
        x = F.gelu(x)
        x = self.c_proj(x)
        return self.dropout(x)


class TransformerBlock(nn.Module):
    """Pre-norm transformer block (identical structure to BabyGPT).

    x = x + Attention(LayerNorm(x))
    x = x + FFN(LayerNorm(x))
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.ln_1 = LayerNorm(config.n_embd, bias=config.bias)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config.n_embd, bias=config.bias)
        self.ffn = FeedForward(config)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.attn(self.ln_1(x))
        x = x + self.ffn(self.ln_2(x))
        return x


# ---------------------------------------------------------------------------
# Full model
# ---------------------------------------------------------------------------

class MicroGPT(nn.Module):
    """Decoder-only transformer language model (GPT-2 124M).

    Compared to BabyGPT:
    - Larger: 12 layers, 12 heads, 768 dim, 1024 context, 32K vocab
    - No weight tying: lm_head has its own parameters (follows GPT-2
      paper)
    - Vocab padding: vocab_size is rounded up to a multiple of 64 for
      tensor-core alignment (the extra embeddings are never used)
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.config = config

        # Round vocab_size up to nearest multiple of 64 for tensor-core
            efficiency
        padded_vocab = ((config.vocab_size + 63) // 64) * 64

        self.transformer = nn.ModuleDict(dict(
            wte=nn.Embedding(padded_vocab, config.n_embd),
            wpe=nn.Embedding(config.sequence_len, config.n_embd),
            drop=nn.Dropout(config.dropout),
            blocks=nn.ModuleList(
                [TransformerBlock(config) for _ in range(config.n_layer)]
```

```
            ),
            ln_f=LayerNorm(config.n_embd, bias=config.bias),
        ))
        self.lm_head = nn.Linear(config.n_embd, padded_vocab, bias=False)

        # No weight tying (unlike BabyGPT).  GPT-2 paper uses separate
        # embedding and output matrices.

        # Initialize weights
        self.apply(self._init_weights)
        # Scaled init for residual projections (GPT-2 convention):
        # scale by 1/sqrt(2 * n_layer) so residual variance stays
           bounded.
        for name, p in self.named_parameters():
            if name.endswith("c_proj.weight"):
                nn.init.normal_(p, mean=0.0, std=0.02 / math.sqrt(2 *
                    config.n_layer))

    def _init_weights(self, module: nn.Module) -> None:
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(
        self,
        tokens: torch.Tensor,
        targets: torch.Tensor | None = None,
    ) -> tuple[torch.Tensor, torch.Tensor | None]:
        B, T = tokens.shape
        assert T <= self.config.sequence_len

        positions = torch.arange(T, device=tokens.device)
        x = self.transformer.wte(tokens) +
           self.transformer.wpe(positions)
        x = self.transformer.drop(x)

        for block in self.transformer.blocks:
            x = block(x)

        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)  # (B, T, padded_vocab)
        # Crop logits to actual vocab_size (remove padding tokens)
        logits = logits[..., :self.config.vocab_size]

        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)),
                targets.view(-1),
            )
        return logits, loss

    @torch.no_grad()
    def generate(
        self,
```

```python
        tokens: torch.Tensor,
        max_new_tokens: int,
        temperature: float = 1.0,
        top_k: int | None = None,
    ) -> torch.Tensor:
        """Autoregressive generation (identical to BabyGPT)."""
        for _ in range(max_new_tokens):
            tokens_cropped = (
                tokens if tokens.size(1) <= self.config.sequence_len
                else tokens[:, -self.config.sequence_len:]
            )
            logits, _ = self(tokens_cropped)
            logits = logits[:, -1, :] / temperature

            if top_k is not None:
                topk_vals, _ = torch.topk(logits, min(top_k,
                    logits.size(-1)))
                logits[logits < topk_vals[:, [-1]]] = float("-inf")

            probs = F.softmax(logits, dim=-1)
            next_token = torch.multinomial(probs, num_samples=1)
            tokens = torch.cat([tokens, next_token], dim=1)

        return tokens

    def count_parameters(self) -> int:
        """Trainable parameters (all are unique --- no weight tying)."""
        return sum(p.numel() for p in self.parameters())
```

Checkpoint: Instantiate the model and verify the parameter count:

Shell — run in your terminal

```shell
uv run python -c "
from microgpt.config import GPTConfig
from microgpt.model import MicroGPT
model = MicroGPT(GPTConfig())
print(f'Parameters: {model.count_parameters():,}')
"
```

Expected output

```
Parameters: 136,071,936
```

2.6 Comparing with BabyGPT

Here is how parameter mass is distributed:

Component	BabyGPT	MicroGPT
Token embedding (`wte`)	24 960	25 165 824
Position embedding (`wpe`)	98 304	786 432
Transformer blocks	10 621 440	84 953 088
Final LayerNorm	384	768
Output head (`lm_head`)	0 (tied)	25 165 824
Total	**10.7M**	**136.1M**

At BabyGPT scale the embedding is negligible (0.2% of total parameters). At GPT-2 scale, the two embedding matrices (token + output) account for 37% of parameters. The transformer blocks — the part that actually "thinks" — account for 62%.

Tip: This is why scaling laws focus on non-embedding parameters. Doubling the vocabulary doubles the embedding cost but does not make the model smarter. Doubling the number of layers or the embedding dimension adds capacity where it matters.

2.7 Summary

We built the canonical GPT-2 124M model by scaling up BabyGPT:

1. **Bigger dimensions**: 12 layers, 12 heads, 768 embedding dim, 1 024 context.
2. **BPE vocabulary**: 32 768 tokens instead of 65 characters.
3. **No weight tying**: separate embedding and output head.
4. **No dropout**: unnecessary with large training sets.
5. **Vocab padding**: round up for tensor-core alignment.

The architecture is identical to BabyGPT — the same pre-norm transformer block with LayerNorm, GELU, and learned positional embeddings. In the next chapter we replace several of these components with modern alternatives that improve training stability, inference efficiency, and model quality.

Chapter 3

Modern Attention Techniques

The GPT-2 architecture we built in the previous chapter is a faithful reproduction of the 2019 paper. Since then, several techniques have become standard practice in production language models. Each one is a self-contained improvement: you can adopt them individually, and each has a clear paper trail.

In this chapter we replace learned positional embeddings with **Rotary Position Embeddings** (RoPE), swap LayerNorm for **RMSNorm**, add **Grouped-Query Attention** (GQA), enable **Flash Attention** via PyTorch's SDPA, switch the FFN activation from GELU to relu^2, and add **QK normalization**.

All changes land in `microgpt/attention.py` (the new attention module) and an updated `microgpt/model.py`.

3.1 RMSNorm

Paper: Zhang & Sennrich, "Root Mean Square Layer Normalization" (2019).

LayerNorm normalizes by subtracting the mean and dividing by the standard deviation:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

RMSNorm drops the mean-centering step. It normalizes by the root mean square only:

$$\text{RMSNorm}(x) = \gamma \cdot \frac{x}{\text{RMS}(x) + \epsilon}, \qquad \text{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}$$

This is simpler (one fewer reduction op, no learnable bias β) and empirically works just as well. LLaMA, Gemma, and most modern LLMs use RMSNorm. PyTorch provides `F.rms_norm` for an optimized implementation.

File: `microgpt/model.py`

```
class RMSNorm(nn.Module):
    """Root Mean Square LayerNorm (Zhang & Sennrich, 2019).

    RMSNorm(x) = x / RMS(x) * gamma
    where RMS(x) = sqrt(mean(x^2) + eps)
```

```
    Compared to LayerNorm, RMSNorm skips the mean-centering step.
    This makes it simpler (fewer ops, no bias parameter) and empirically
    works just as well.  Used by LLaMA, Gemma, and most modern LLMs.
    """

    def __init__(self, n_embd: int, eps: float = 1e-6) -> None:
        super().__init__()
        self.weight = nn.Parameter(torch.ones(n_embd))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return F.rms_norm(x, self.weight.shape, self.weight, self.eps)
```

3.2 Rotary Position Embeddings (RoPE)

Paper: Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (2021).

3.2.1 The problem with learned positional embeddings

BabyGPT and canonical GPT-2 add a learned position vector to each token:

$$x_t = \text{token_emb}(t) + \text{pos_emb}(t)$$

This has two limitations:

1. **Fixed maximum length.** The position embedding table has `sequence_len` rows. The model cannot handle sequences longer than what it was trained on.
2. **Absolute positions.** Position 5 always gets the same vector, regardless of context. The model has no built-in notion of "3 tokens apart."

3.2.2 The RoPE idea

RoPE encodes position by *rotating* query and key vectors in 2D subspaces. Instead of adding a position vector, it multiplies by a rotation matrix.

Given a head dimension d, pair up the dimensions into $d/2$ pairs. Each pair (x_{2i}, x_{2i+1}) is a 2D vector. At position m, rotate this pair by angle $m \cdot \theta_i$:

$$\begin{pmatrix} y_{2i} \\ y_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix}$$

where $\theta_i = 1/10000^{2i/d}$.

The key insight: when computing $q_m^T k_n$ (the attention score between positions m and n), the rotations combine as $\cos((m-n)\theta_i)$ — the score depends only on the *relative* distance $m - n$. This gives the model relative position awareness with zero learned parameters.

3.2.3 Implementation

We precompute the cos and sin tables once and apply them in the attention forward pass.

File: `microgpt/attention.py`

```python
"""Modern attention components: RoPE, GQA, QK-norm, Flash Attention.

This module replaces the canonical GPT-2 attention (Chapter 2) with
techniques that have become standard since 2019.  Each is a
    self-contained
improvement with a clear paper trail.
"""

import math

import torch
import torch.nn as nn
import torch.nn.functional as F


#
    ---------------------------------------------------------------------------
# Rotary Position Embeddings (RoPE) --- Su et al. 2021
#
    ---------------------------------------------------------------------------
# Instead of adding a learned position vector to each token, RoPE encodes
# relative position by *rotating* query and key vectors in 2D subspaces.
#
# Given a head-dim d, we pair up the dimensions into d/2 pairs.
# Each pair (x_{2i}, x_{2i+1}) is treated as a 2D vector and rotated
# by an angle theta_i * position, where theta_i = 1 / 10000^{2i/d}.
#
# The rotation matrix for pair i at position m is:
#
#   [ cos(m*theta_i)  -sin(m*theta_i) ]   [ x_{2i}   ]
#   [ sin(m*theta_i)   cos(m*theta_i) ] * [ x_{2i+1} ]
#
# This means: attention between positions m and n depends only on the
# *difference* (m - n), giving the model relative position awareness
# without any learned parameters.


def precompute_rope_frequencies(
    head_dim: int,
    max_seq_len: int,
    theta: float = 10000.0,
    device: torch.device | None = None,
) -> tuple[torch.Tensor, torch.Tensor]:
    """Precompute cos/sin tables for RoPE.

    Returns (cos, sin) each of shape (1, max_seq_len, 1, head_dim/2).
    The singleton dims broadcast over batch and heads.
    """
    assert head_dim % 2 == 0
    # theta_i = 1 / 10000^{2i/d} for i = 0, 1, ..., d/2 - 1
    half = head_dim // 2
    freqs = 1.0 / (theta ** (torch.arange(0, head_dim, 2,
        device=device).float() / head_dim))
    # positions: 0, 1, ..., max_seq_len - 1
    positions = torch.arange(max_seq_len, device=device).float()
    # outer product: (max_seq_len, head_dim/2)
    angles = torch.outer(positions, freqs)
```

```
    # Shape: (1, max_seq_len, 1, d/2) --- broadcasts over (B, T, H, D/2)
    cos = torch.cos(angles).unsqueeze(0).unsqueeze(2)  # (1,
        max_seq_len, 1, d/2)
    sin = torch.sin(angles).unsqueeze(0).unsqueeze(2)
    return cos, sin
```

The application function splits each head into two halves and rotates:

File: microgpt/attention.py

```
def apply_rotary_emb(
    x: torch.Tensor,
    cos: torch.Tensor,
    sin: torch.Tensor,
) -> torch.Tensor:
    """Apply RoPE rotation to x.

    x: (B, T, H, D)  where D = head_dim
    cos, sin: (1, T, 1, D/2)  --- precomputed tables
    """
    d = x.shape[-1] // 2
    x1, x2 = x[..., :d], x[..., d:]
    # Rotation: [x1, x2] -> [x1*cos + x2*sin, -x1*sin + x2*cos]
    # Wait --- the standard rotation is [x1*cos - x2*sin, x1*sin +
        x2*cos].
    # The nanochat convention (which we follow) swaps the sign pattern.
    # Both are valid --- they just rotate in opposite directions.
    y1 = x1 * cos + x2 * sin
    y2 = x1 * (-sin) + x2 * cos
    return torch.cat([y1, y2], dim=-1)
```

Tip: There are two common implementations of RoPE. The "split" version (used here and by nanochat) puts the first $d/2$ dims in one group and the second $d/2$ in another. The "interleaved" version (used by LLaMA) pairs up even and odd indices. Both produce the same rotary behavior — the dot product between rotated q and k depends on relative position regardless of which pairing you use.

3.3 Grouped-Query Attention (GQA)

Paper: Ainslie et al., "GQA: Training Generalized Multi-Query Transformers from Multi-Head Checkpoints" (2023).

Standard multi-head attention (MHA) gives each head its own Q, K, and V projections. With 12 heads, that means 12 independent sets of K and V matrices.

GQA reduces the number of K/V heads while keeping the full number of Q heads. Groups of query heads share one K/V head:

Method	Q heads	KV heads	KV memory
MHA	12	12	1x
GQA (4 KV)	12	4	0.33x
MQA	12	1	0.08x

The main benefit is **inference efficiency**: the KV cache (Chapter 7) shrinks proportionally to the number of KV heads. With GQA-4 instead of MHA, the KV cache is 3x smaller, enabling longer sequences or larger batch sizes.

In the implementation, we project K and V to fewer dimensions, then expand them to match Q during the attention computation:

File: `microgpt/attention.py`

```
class CausalSelfAttention(nn.Module):
    """Modern causal self-attention.

    Improvements over the canonical GPT-2 attention:
    - RoPE instead of learned positional embeddings
    - Grouped-Query Attention (GQA): n_kv_head <= n_head
    - QK normalization for training stability
    - PyTorch SDPA (Flash Attention / memory-efficient backend)
    """

    def __init__(
        self,
        n_embd: int,
        n_head: int,
        n_kv_head: int,
        bias: bool = False,
    ) -> None:
        super().__init__()
        assert n_embd % n_head == 0
        assert n_head % n_kv_head == 0

        self.n_head = n_head
        self.n_kv_head = n_kv_head
        self.head_dim = n_embd // n_head
        self.n_kv_groups = n_head // n_kv_head  # queries per KV head

        # Q projects to full n_head * head_dim; K and V project to
            n_kv_head * head_dim
        self.W_Q = nn.Linear(n_embd, n_head * self.head_dim, bias=bias)
        self.W_K = nn.Linear(n_embd, n_kv_head * self.head_dim,
            bias=bias)
        self.W_V = nn.Linear(n_embd, n_kv_head * self.head_dim,
            bias=bias)
        self.c_proj = nn.Linear(n_embd, n_embd, bias=bias)

    def forward(
        self,
        x: torch.Tensor,
        cos: torch.Tensor,
        sin: torch.Tensor,
    ) -> torch.Tensor:
        B, T, C = x.shape

        # Project to Q, K, V
        # Q: (B, T, n_head, head_dim)
        # K, V: (B, T, n_kv_head, head_dim)
        q = self.W_Q(x).view(B, T, self.n_head, self.head_dim)
        k = self.W_K(x).view(B, T, self.n_kv_head, self.head_dim)
        v = self.W_V(x).view(B, T, self.n_kv_head, self.head_dim)
```

```
        # Apply RoPE to queries and keys
        # cos/sin are (1, max_seq_len, 1, D/2); slice to (1, T, 1, D/2)
        q = apply_rotary_emb(q, cos[:, :T], sin[:, :T])
        k = apply_rotary_emb(k, cos[:, :T], sin[:, :T])

        # QK normalization: normalize Q and K after RoPE
        # Prevents attention logits from growing with depth.
        q = F.rms_norm(q, (self.head_dim,))
        k = F.rms_norm(k, (self.head_dim,))

        # Transpose to (B, H, T, D) for SDPA
        q = q.transpose(1, 2)  # (B, n_head, T, head_dim)
        k = k.transpose(1, 2)  # (B, n_kv_head, T, head_dim)
        v = v.transpose(1, 2)  # (B, n_kv_head, T, head_dim)

        # GQA: expand K and V heads to match Q heads
        # Each group of (n_head / n_kv_head) query heads shares one KV
           head.
        if self.n_kv_groups > 1:
            k = k.unsqueeze(2).expand(-1, -1, self.n_kv_groups, -1, -1)
            k = k.reshape(B, self.n_head, T, self.head_dim)
            v = v.unsqueeze(2).expand(-1, -1, self.n_kv_groups, -1, -1)
            v = v.reshape(B, self.n_head, T, self.head_dim)

        # Flash Attention via SDPA (automatically selects the best
           backend)
        out = F.scaled_dot_product_attention(q, k, v, is_causal=True)

        # Reassemble heads: (B, n_head, T, head_dim) -> (B, T, n_embd)
        out = out.transpose(1, 2).contiguous().view(B, T, C)
        return self.c_proj(out)
```

3.4 QK Normalization

Papers: Henry et al. (2020); Dehghani et al. (2023).

In deep transformers, attention logits ($q \cdot k/\sqrt{d}$) can grow large because the norms of q and k are unbounded. Large logits push softmax toward a one-hot distribution, choking gradient flow.

QK normalization applies RMSNorm to queries and keys after projection (and after RoPE in our case). This bounds the logit magnitude, stabilizing training without changing the attention mechanism.

The implementation is two lines in the attention forward method:

```
q = F.rms_norm(q, (self.head_dim,))
k = F.rms_norm(k, (self.head_dim,))
```

3.5 Flash Attention

Paper: Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness" (2022).

Standard attention materializes the full $T \times T$ attention matrix, requiring $O(T^2)$ memory. Flash Attention computes the *same* result using a tiled algorithm that never materializes the full matrix, reducing memory from $O(T^2)$ to $O(T)$ and running 2-4x faster through better GPU memory hierarchy utilization.

We do not implement Flash Attention ourselves. PyTorch's `F.scaled_dot_product_attention` (SDPA) automatically dispatches to the best available backend:

- **FlashAttention v2** on Ampere+ GPUs (A100, RTX 3090, etc.)
- **Memory-efficient attention** (xformers) as fallback
- **Math-mode** (naive) as a last resort

The one-line call replaces our entire manual attention computation (scale, mask, softmax, dropout, matmul):

```
out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

Tip: If you are running on an H100 or newer GPU, you can install FlashAttention 3 (`flash-attn`) for even faster attention. Nanochat uses FA3 when available and falls back to SDPA otherwise. For this tutorial, SDPA is sufficient and works everywhere.

3.6 relu-squared Activation

Paper: So et al., "Primer: Searching for Efficient Transformers for Language Modeling" (2021).

The GPT-2 FFN uses GELU:

$$\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x)$$

We replace GELU with relu^2:

$$\text{FFN}(x) = W_2 \cdot \text{relu}(W_1 x)^2$$

Why?

- **Sparsity.** ReLU zeros out negative activations. Squaring keeps them zero but amplifies positive activations, producing a naturally sparse hidden representation.
- **Simplicity.** No special function — just `F.relu(x).square()`.
- **Performance.** Empirically competitive with GELU at GPT-2 scale.

File: `microgpt/model.py`

```
class FeedForward(nn.Module):
    """Position-wise FFN with relu-squared activation (So et al., 2021).

    FFN(x) = W2 * relu(W1 * x)^2

    relu-squared is computationally simpler than GELU and produces sparse
    activations (many exact zeros plus a few strongly positive values).
    Empirically competitive with GELU at GPT-2 scale and above.
    """
```

```
    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd,
            bias=config.bias)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.c_fc(x)
        x = F.relu(x).square()  # relu^2
        x = self.c_proj(x)
        return x
```

3.7 Putting It All Together

Here is the updated model. Compared to Chapter 2's canonical GPT-2:

- `LayerNorm` → `RMSNorm`
- Learned `wpe` → RoPE (no position embedding table)
- Standard MHA → GQA with QK-norm and SDPA
- GELU → relu-squared
- Added `embed_norm`: RMSNorm after token embedding (stabilizes early training)

File: `microgpt/model.py`

```
class TransformerBlock(nn.Module):
    """Pre-norm transformer block with modern components.

    x = x + Attention(RMSNorm(x))
    x = x + FFN(RMSNorm(x))
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.ln_1 = RMSNorm(config.n_embd)
        self.attn = CausalSelfAttention(
            n_embd=config.n_embd,
            n_head=config.n_head,
            n_kv_head=config.n_kv_head,
            bias=config.bias,
        )
        self.ln_2 = RMSNorm(config.n_embd)
        self.ffn = FeedForward(config)

    def forward(
        self,
        x: torch.Tensor,
        cos: torch.Tensor,
        sin: torch.Tensor,
    ) -> torch.Tensor:
        x = x + self.attn(self.ln_1(x), cos, sin)
        x = x + self.ffn(self.ln_2(x))
        return x
```

File: microgpt/model.py

```python
class MicroGPT(nn.Module):
    """Decoder-only transformer language model (GPT-2 scale, modern
       arch).

    Key differences from the canonical GPT-2 (Chapter 2):
    - RMSNorm instead of LayerNorm
    - RoPE instead of learned positional embeddings (no wpe)
    - GQA support (n_kv_head can be < n_head)
    - QK normalization inside attention
    - relu-squared activation instead of GELU
    - Flash Attention via SDPA
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.config = config

        # Pad vocab to nearest multiple of 64 for tensor-core efficiency
        padded_vocab = ((config.vocab_size + 63) // 64) * 64

        self.transformer = nn.ModuleDict(dict(
            wte=nn.Embedding(padded_vocab, config.n_embd),
            blocks=nn.ModuleList(
                [TransformerBlock(config) for _ in range(config.n_layer)]
            ),
            ln_f=RMSNorm(config.n_embd),
        ))
        self.lm_head = nn.Linear(config.n_embd, padded_vocab, bias=False)

        # No positional embedding table --- RoPE handles positions
        # Precompute RoPE cos/sin tables
        head_dim = config.n_embd // config.n_head
        cos, sin = precompute_rope_frequencies(
            head_dim, config.sequence_len, device=None,
        )
        self.register_buffer("rope_cos", cos)
        self.register_buffer("rope_sin", sin)

        # RMSNorm after token embedding (stabilizes early training)
        self.embed_norm = RMSNorm(config.n_embd)

        # Initialize weights
        self.apply(self._init_weights)
        # Scaled init for residual projections
        for name, p in self.named_parameters():
            if name.endswith("c_proj.weight"):
                nn.init.normal_(p, mean=0.0, std=0.02 / math.sqrt(2 *
                    config.n_layer))

    def _init_weights(self, module: nn.Module) -> None:
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

```python
    def forward(
        self,
        tokens: torch.Tensor,
        targets: torch.Tensor | None = None,
    ) -> tuple[torch.Tensor, torch.Tensor | None]:
        B, T = tokens.shape
        assert T <= self.config.sequence_len

        # Token embeddings only --- no positional embeddings (RoPE
            handles this)
        x = self.transformer.wte(tokens)
        x = self.embed_norm(x)

        # Pass cos/sin through the blocks for RoPE
        cos = self.rope_cos.to(x.dtype)
        sin = self.rope_sin.to(x.dtype)

        for block in self.transformer.blocks:
            x = block(x, cos, sin)

        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)
        # Crop logits to actual vocab_size (remove padding tokens)
        logits = logits[..., :self.config.vocab_size]

        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.reshape(-1, logits.size(-1)),
                targets.reshape(-1),
            )
        return logits, loss
```

The `generate()` method is unchanged from Chapter 2 — Chapter 7 will upgrade it with a KV cache.

Checkpoint: Verify the modernized model:

Shell — run in your terminal

```shell
uv run python -c "
from microgpt.config import GPTConfig
from microgpt.model import MicroGPT
import torch
# MHA (default): 12 query heads, 12 KV heads
model = MicroGPT(GPTConfig())
print(f'MHA params: {model.count_parameters():,}')
# GQA: 12 query heads, 4 KV heads
model_gqa = MicroGPT(GPTConfig(n_kv_head=4))
print(f'GQA params: {model_gqa.count_parameters():,}')
x = torch.randint(0, 32768, (2, 64))
logits, loss = model(x[:, :-1], x[:, 1:])
print(f'Forward OK: loss {loss.item():.4f}')
"
```

Expected output

```
MHA params: 135,286,272
GQA params: 125,849,088
Forward OK: loss 10.59
```

GQA with 4 KV heads saves about 10M parameters (the K and V projections shrink from 768×768 to 768×256 per block). The bigger win comes at inference when the KV cache is 3x smaller.

Tip: Nanochat includes several more experimental techniques beyond what we cover here: value embeddings (ResFormer-style), per-layer residual scaling lambdas, smear gates, and the Muon optimizer. These are active areas of research with less established paper trails. We teach the techniques that have become industry-standard (RoPE, RMSNorm, GQA, Flash Attention) and leave the experimental ones as pointers for further reading.

3.8 Summary

We upgraded every major component of the GPT-2 architecture:

Component	Canonical GPT-2	MicroGPT (modern)
Normalization	LayerNorm	RMSNorm
Position encoding	Learned embeddings	RoPE
Attention	Standard MHA	GQA + QK-norm + SDPA
FFN activation	GELU	relu-squared

Each change is well-motivated, well-documented in the literature, and independently beneficial. The model is now ready for large-scale training — which requires a data pipeline (next chapter) and distributed computing infrastructure (Chapter 5).

Chapter 4

Data Pipeline

BabyGPT trains on a single 1 MB text file that fits in RAM. GPT-2 was trained on WebText, a 40 GB corpus of web pages. Modern reproductions use even larger, openly available datasets like FineWeb-Edu. In this chapter we build a data pipeline that downloads shards in Parquet format, tokenizes documents with BOS prepended, packs them efficiently into fixed-length sequences using best-fit bin packing, and distributes batches across multiple GPUs.

4.1 Web-Scale Datasets

The original GPT-2 was trained on **WebText**, a custom dataset of web pages filtered by Reddit upvotes (links with ≥ 3 karma). WebText was never publicly released. Several open alternatives now exist:

Dataset	Size	Description
The Pile	800 GB	22 diverse sources (academic, books, code, web)
C4	750 GB	Cleaned Common Crawl
FineWeb	15 TB	Cleaned Common Crawl (2013–2024)
FineWeb-Edu	1.3 TB	FineWeb filtered for educational content
ClimbMix	400B tokens	Nanochat's curated multi-source blend

We use **FineWeb-Edu**: it is openly available on HuggingFace, well-documented, and filtered for quality. For a GPT-2 scale reproduction ($\sim$10B tokens), we download a small subset (10–50 shards, each $\sim$100M tokens).

4.2 The Parquet Format

FineWeb-Edu is distributed as Apache Parquet files. Parquet is a columnar storage format designed for large-scale data:

- **Columnar**: data is stored column-by-column, so reading just the `text` column skips all metadata columns.
- **Compressed**: Snappy or Zstandard compression reduces file sizes by 2–5x.
- **Row groups**: each file is divided into row groups (typically 10K–100K rows each). You can read one row group at a time without loading the entire file.

The `pyarrow` library provides fast Parquet reading in Python.

4.3 Dataset Management

The `microgpt/dataset.py` module handles downloading and listing parquet shards:

File: `microgpt/dataset.py`

```
"""Dataset management: download and list FineWeb-Edu parquet shards.

FineWeb-Edu is an open, high-quality web text dataset filtered for
educational content.  It is distributed as parquet files on HuggingFace.
Each file contains a 'text' column with document strings.

Usage:
    python -m microgpt.dataset                  # download 10 shards (~5
        GB)
    python -m microgpt.dataset --num-shards 50  # download more for full
        training
"""

import os
import glob

import requests


#
    ---------------------------------------------------------------------------
# Configuration
#
    ---------------------------------------------------------------------------

DATA_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)),
    "data", "microgpt")

# FineWeb-Edu-score-2 sample: ~10B tokens total, split into 100 parquet
    files
# Each shard is ~50-100 MB, containing ~100M tokens
BASE_URL = (
    "https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu-score-2"
    "/resolve/main/data/CC-MAIN-2024-10"
)
# Shard filenames: train-00000-of-00099.parquet ...
    train-00098-of-00099.parquet
NUM_SHARDS_AVAILABLE = 99


def shard_filename(idx: int) -> str:
    return f"train-{idx:05d}-of-{NUM_SHARDS_AVAILABLE:05d}.parquet"


def shard_path(idx: int) -> str:
    return os.path.join(DATA_DIR, shard_filename(idx))


#
    ---------------------------------------------------------------------------
# Download
#
    ---------------------------------------------------------------------------
```

```python
def download_shard(idx: int) -> None:
    """Download a single parquet shard if not already present."""
    path = shard_path(idx)
    if os.path.exists(path):
        return
    url = f"{BASE_URL}/{shard_filename(idx)}"
    print(f"  Downloading shard {idx}: {url}")
    os.makedirs(DATA_DIR, exist_ok=True)
    response = requests.get(url, stream=True, timeout=60)
    response.raise_for_status()
    # Write to a temp file first, then rename (atomic on POSIX)
    tmp_path = path + ".tmp"
    with open(tmp_path, "wb") as f:
        for chunk in response.iter_content(chunk_size=8 * 1024 * 1024):
            f.write(chunk)
    os.rename(tmp_path, path)


def download_shards(num_shards: int = 10) -> None:
    """Download parquet shards for training."""
    num_shards = min(num_shards, NUM_SHARDS_AVAILABLE)
    print(f"Downloading {num_shards} FineWeb-Edu shards to {DATA_DIR}")
    for idx in range(num_shards):
        download_shard(idx)
    print(f"Done. {num_shards} shards in {DATA_DIR}")


# ---------------------------------------------------------------------------
# Listing
# ---------------------------------------------------------------------------

def list_parquet_files() -> list[str]:
    """Return sorted list of all downloaded parquet shard paths."""
    pattern = os.path.join(DATA_DIR, "train-*.parquet")
    paths = sorted(glob.glob(pattern))
    return paths
```

Checkpoint: Download two shards and verify:

Shell — run in your terminal

```shell
uv run python -m microgpt.dataset --num-shards 2
```

4.4 BOS-Aligned Best-Fit Packing

4.4.1 The packing problem

Documents have variable lengths. Training requires fixed-length sequences of $T+1$ tokens (input + shifted target). How do we fill each row?

Naive concatenation glues documents end-to-end and chops the result into T-length chunks. This is simple and wastes nothing, but a model token at position 500 might attend back to a

BOS token at position 200 that belongs to a completely different document. The attention is wasted on irrelevant context.

BOS-aligned packing ensures every row starts with a BOS token and that each token can attend back to its own document's beginning. The tradeoff: documents that don't fit a row are cropped, wasting ∼35% of tokens.

4.4.2 The best-fit algorithm

For each row in the batch:

1. Maintain a buffer of tokenized documents.
2. Find the **largest** document that fits entirely in the remaining space. Append it to the row. Repeat.
3. When no document fits, **crop** the shortest document in the buffer to fill the remaining space exactly.

This achieves 100% utilization (no padding) while minimizing the amount of cropped text by preferring documents that fit without cropping.

4.5 The Distributed Dataloader

File: `microgpt/dataloader.py`

```
"""Distributed dataloader with BOS-aligned best-fit packing.

This dataloader:
1. Reads documents from parquet shards (via dataset.py)
2. Tokenizes them with BOS prepended
3. Packs documents into fixed-length rows using best-fit bin packing
4. Distributes shards across DDP ranks
5. Pre-allocates pinned CPU buffers for efficient GPU transfer

Every row starts with a BOS token, so the model always sees the beginning
of at least one document.  Documents that don't fit are cropped (the
    cropped
portion is discarded, not carried over).  This wastes ~35% of tokens but
ensures every token can attend back to its document's BOS.
"""

import torch
import pyarrow.parquet as pq

from .dataset import list_parquet_files


#
    ------------------------------------------------------------------------
# Document iterator (with DDP sharding)
#
    ------------------------------------------------------------------------

def _document_batches(
    split: str,
    rank: int = 0,
    world_size: int = 1,
```

```
    batch_size: int = 128,
):
    """Infinite iterator over batches of document strings from parquet
       files.

    DDP sharding: each rank reads every world_size-th row group, so ranks
    see disjoint data without any communication.

    Yields: (text_batch, shard_idx)
    """
    paths = list_parquet_files()
    assert paths, "No parquet files found. Run: python -m
       microgpt.dataset"
    # Last shard is validation, rest are training
    if split == "train":
        paths = paths[:-1]
    else:
        paths = paths[-1:]

    while True:  # infinite iteration (multi-epoch)
        for shard_idx, filepath in enumerate(paths):
            pf = pq.ParquetFile(filepath)
            # Each rank reads every world_size-th row group
            rg_idx = rank
            while rg_idx < pf.num_row_groups:
                rg = pf.read_row_group(rg_idx)
                texts = rg.column("text").to_pylist()
                for i in range(0, len(texts), batch_size):
                    yield texts[i : i + batch_size], shard_idx
                rg_idx += world_size
```

DDP sharding is simple: each rank reads every N^{th} row group (where N is the world size). Rank 0 reads row groups $0, N, 2N, \ldots$; rank 1 reads $1, N+1, 2N+1, \ldots$. No communication is needed — each rank sees disjoint data.

File: microgpt/dataloader.py

```
def _pack_row(
    doc_buffer: list[list[int]],
    row: torch.Tensor,
    row_capacity: int,
) -> None:
    """Pack documents from doc_buffer into a single row using best-fit.

    Algorithm:
    1. Find the largest document that fits entirely in the remaining
       space.
    2. Append it and repeat.
    3. When no document fits, crop the shortest one to fill exactly.

    This achieves 100% utilization (no padding) at the cost of ~35%
       cropped
    tokens.
    """
    pos = 0
    while pos < row_capacity:
        remaining = row_capacity - pos
```

```
        # Find largest doc that fits entirely
        best_idx = -1
        best_len = 0
        for i, doc in enumerate(doc_buffer):
            doc_len = len(doc)
            if doc_len <= remaining and doc_len > best_len:
                best_idx = i
                best_len = doc_len

        if best_idx >= 0:
            doc = doc_buffer.pop(best_idx)
            row[pos : pos + len(doc)] = torch.tensor(doc,
               dtype=torch.long)
            pos += len(doc)
        else:
            # No doc fits --- crop shortest to fill remaining space
               exactly
            shortest_idx = min(
                range(len(doc_buffer)), key=lambda i: len(doc_buffer[i])
            )
            doc = doc_buffer.pop(shortest_idx)
            row[pos : pos + remaining] = torch.tensor(
                doc[:remaining], dtype=torch.long
            )
            pos += remaining
```

File: microgpt/dataloader.py

```
def create_dataloader(
    tokenizer,
    batch_size: int,
    sequence_len: int,
    split: str = "train",
    rank: int = 0,
    world_size: int = 1,
    device: str = "cuda",
    buffer_size: int = 1000,
):
    """Create an infinite dataloader that yields (inputs, targets)
       batches.

    Args:
        tokenizer: BPETokenizer instance (needs .encode_with_bos and
           .bos_id)
        batch_size: number of rows per batch (per GPU)
        sequence_len: context length (T)
        split: "train" or "val"
        rank: DDP rank (0 for single-GPU)
        world_size: DDP world size (1 for single-GPU)
        device: target device ("cuda" or "cpu")
        buffer_size: min documents in buffer before packing

    Yields:
        (inputs, targets): both shape (batch_size, sequence_len), on
           device
    """
```

```
    row_capacity = sequence_len + 1  # +1 because targets are shifted by
        1
    batches = _document_batches(split, rank, world_size)
    doc_buffer: list[list[int]] = []

    # Pre-allocate buffers for efficient transfer
    use_cuda = device == "cuda"
    row_buffer = torch.empty(
        (batch_size, row_capacity), dtype=torch.long
    )
    cpu_buffer = torch.empty(
        2 * batch_size * sequence_len, dtype=torch.long,
           pin_memory=use_cuda
    )
    gpu_buffer = torch.empty(
        2 * batch_size * sequence_len, dtype=torch.long, device=device
    )
    # Views into the flat buffers
    cpu_inputs = cpu_buffer[: batch_size * sequence_len].view(
        batch_size, sequence_len
    )
    cpu_targets = cpu_buffer[batch_size * sequence_len :].view(
        batch_size, sequence_len
    )
    inputs = gpu_buffer[: batch_size * sequence_len].view(
        batch_size, sequence_len
    )
    targets = gpu_buffer[batch_size * sequence_len :].view(
        batch_size, sequence_len
    )

    while True:
        # Build a full batch
        for row_idx in range(batch_size):
            # Refill buffer if needed
            while len(doc_buffer) < buffer_size:
                text_batch, _ = next(batches)
                for text in text_batch:
                    doc_ids = tokenizer.encode_with_bos(text)
                    doc_buffer.append(doc_ids)

            _pack_row(doc_buffer, row_buffer[row_idx], row_capacity)

        # Split into inputs and targets (shifted by 1)
        cpu_inputs.copy_(row_buffer[:, :-1])
        cpu_targets.copy_(row_buffer[:, 1:])

        # Single host-to-device transfer
        gpu_buffer.copy_(cpu_buffer, non_blocking=use_cuda)
        yield inputs, targets
```

4.5.1 Buffer management

The dataloader maintains a **document buffer** of at least `buffer_size` tokenized documents. When the buffer runs low, it reads the next batch of raw text from parquet, tokenizes it (prepending BOS), and appends the token lists to the buffer.

Pre-allocated pinned CPU buffers and a single host-to-device copy per batch minimize transfer overhead.

Tip: Why is the row capacity $T + 1$ instead of T? Because the training targets are the inputs shifted by one position. A row of $T + 1$ tokens gives T (input, target) pairs: input = row[0:T], target = row[1:T+1]. This is the same teacher-forcing setup as BabyGPT, just with more careful buffer management.

4.6 Summary

The data pipeline has three components:

1. **Dataset manager** (`microgpt/dataset.py`): downloads parquet shards from HuggingFace and lists them.
2. **Document iterator**: reads row groups from parquet files, sharded across DDP ranks, with infinite looping.
3. **BOS-aligned dataloader** (`microgpt/dataloader.py`): tokenizes documents, packs them into fixed-length rows using best-fit bin packing, and transfers batches to GPU.

With the data pipeline in place, the next chapter sets up the distributed training infrastructure (DDP, mixed precision, gradient accumulation) needed to actually train on this data.

Chapter 5

Distributed Training

Training 135 million parameters on billions of tokens takes thousands of GPU-hours. A single GPU can do it, but an 8-GPU node cuts wall-clock time by nearly 8×. In this chapter we introduce PyTorch's **Distributed Data Parallel** (DDP), set up multi-GPU training with `torchrun`, implement gradient accumulation for large effective batch sizes, and adopt mixed-precision arithmetic to double throughput.

5.1 Why Distributed?

A rough estimate of the compute required to train GPT-2 124M:

$$C \approx 6 \cdot N \cdot D$$

where N is the parameter count and D is the number of training tokens. For $N = 135\text{M}$ and $D = 10\text{B}$ tokens:

$$C \approx 6 \times 135\text{M} \times 10\text{B} = 8.1 \times 10^{18} \text{ FLOPs}$$

An A100 GPU delivers ∼312 TFLOPS in bf16. At 40% utilization (a realistic MFU):

- 1 GPU: ∼65 hours
- 8 GPUs: ∼8 hours

The factor-of-2 comes from memory-bound operations and communication overhead, but 8× GPU scaling is nearly linear for DDP at this model size.

5.2 Data Parallelism (DDP)

DDP is the simplest form of distributed training. Each GPU has a **complete copy** of the model. Each processes a different batch of data. After the backward pass, gradients are **averaged** across all GPUs using an all-reduce operation:

1. Each rank computes ∇L_i on its local batch.
2. All-reduce: $\bar{\nabla L} = \frac{1}{W} \sum_{i=0}^{W-1} \nabla L_i$ (NCCL handles this).
3. Each rank updates its model with $\bar{\nabla L}$.

Since every rank starts with the same weights and applies the same averaged gradient, the models stay in sync. DDP is mathematically equivalent to training on a single GPU with a $W\times$ larger batch.

5.2.1 Process groups and ranks

`torchrun` sets three environment variables:

- `RANK`: global rank (0 to $W - 1$).
- `LOCAL_RANK`: rank within the current node (0 to GPUs-per-node -1).
- `WORLD_SIZE`: total number of GPUs (W).

Our `distributed.py` reads these and initializes the NCCL process group:

File: `microgpt/distributed.py`

```python
"""Distributed training utilities for DDP.

This module provides helpers for setting up and tearing down PyTorch
Distributed Data Parallel (DDP) training with NCCL backend.

Usage:
    rank, local_rank, world_size = ddp_init()
    # ... training loop ...
    ddp_cleanup()
"""

import os

import torch
import torch.distributed as dist


def ddp_init() -> tuple[int, int, int]:
    """Initialize DDP and return (rank, local_rank, world_size).

    Reads RANK, LOCAL_RANK, WORLD_SIZE from environment (set by
        torchrun).
    If these are not set, assumes single-GPU mode.
    """
    if "RANK" not in os.environ:
        # Single-GPU mode
        return 0, 0, 1

    rank = int(os.environ["RANK"])
    local_rank = int(os.environ["LOCAL_RANK"])
    world_size = int(os.environ["WORLD_SIZE"])

    dist.init_process_group(backend="nccl")
    torch.cuda.set_device(local_rank)

    return rank, local_rank, world_size


def ddp_cleanup() -> None:
    """Destroy the DDP process group if it exists."""
    if dist.is_initialized():
        dist.destroy_process_group()


def is_master(rank: int) -> bool:
    """Return True if this is the master process (rank 0)."""
```

```
    return rank == 0


def print0(*args, **kwargs) -> None:
    """Print only on rank 0 (or when not using DDP)."""
    if "RANK" not in os.environ or int(os.environ["RANK"]) == 0:
        print(*args, **kwargs)
```

5.3 Gradient Accumulation

Sometimes the effective batch size we want (B_{eff}) is larger than what fits in GPU memory. Gradient accumulation splits each optimizer step into multiple forward-backward passes (micro-steps), accumulating gradients before stepping:

$$B_{\text{eff}} = B_{\text{device}} \times W \times A$$

where B_{device} is the per-GPU micro-batch, W is the world size, and A is the number of accumulation steps.

In practice, we set a target in tokens:

```
TOTAL_BATCH_TOKENS = 524_288   # ~0.5M tokens per optimizer step
tokens_per_step = DEVICE_BATCH_SIZE * sequence_len
grad_accum_steps = TOTAL_BATCH_TOKENS // (tokens_per_step * world_size)
```

During accumulation, we skip DDP gradient synchronization on all micro-steps except the last (using `model.no_sync()`). This avoids unnecessary all-reduce overhead.

5.4 Mixed Precision Training

Modern GPUs have specialized tensor cores that compute matrix multiplications in reduced precision (bf16 or fp16) at 2x the throughput of fp32.

BFloat16 is preferred over float16 because it has the same exponent range as fp32 (8 exponent bits), eliminating the need for a gradient scaler. Training is simple:

```
with torch.amp.autocast(device_type, dtype=torch.bfloat16):
    _, loss = model(inputs, targets)
    loss = loss / grad_accum_steps
loss.backward()
```

Weights remain in fp32 (the "master weights") for optimizer precision. Only activations and matmuls run in bf16.

Tip: Nanochat goes further: it defines a custom `Linear` class that casts weights to bf16 in the forward pass, keeping master weights in fp32 without using `autocast`. This is slightly more explicit but achieves the same result. We use `autocast` for simplicity.

5.5 torch.compile

PyTorch 2.0 introduced `torch.compile`, which fuses operations, eliminates Python overhead, and generates optimized GPU kernels:

```
model = torch.compile(model)
```

This single line typically gives 10–30% speedup. The first iteration is slow (compilation), but subsequent iterations are faster.

5.6 Summary

The distributed training infrastructure consists of:

1. **DDP**: each GPU holds a full model copy; gradients are all-reduced.
2. **Gradient accumulation**: large effective batch sizes without more memory.
3. **Mixed precision**: bf16 activations for 2x throughput, fp32 master weights.
4. **torch.compile**: fused kernels for 10–30% speedup.

In the next chapter we assemble the complete training script and cover the optimizer, LR schedule, monitoring, and deployment on rented GPU clusters.

Chapter 6

Training at Scale

With distributed infrastructure in place, this chapter focuses on the training recipe: the optimizer, learning-rate schedule, batch-size selection, gradient clipping, and monitoring. We assemble the complete training script, show how to checkpoint and resume, and provide a practical guide to launching runs on rented GPU clusters.

6.1 The AdamW Optimizer

We use AdamW (Loshchilov & Hutter, 2019) with decoupled weight decay. The key insight from BabyGPT carries over: separate parameters into two groups:

- **Decay group**: all 2D+ parameters (weight matrices). Apply weight decay $\lambda = 0.1$.
- **No-decay group**: all 1D parameters (biases, LayerNorm/RMSNorm weights, embedding weights). No weight decay.

Weight decay on norms and biases is harmful because these parameters control scale and shift, not capacity.

We also enable the `fused=True` option for a faster CUDA kernel.

6.2 Learning Rate Schedule

6.2.1 Warmup + cosine decay

Same structure as BabyGPT but different hyperparameters:

1. **Linear warmup**: 0 to lr_{max} over 200 steps. Prevents large early gradients from destabilizing training.
2. **Cosine decay**: lr_{max} to lr_{min} over the remaining steps.

$$\text{lr}(t) = \begin{cases} \text{lr}_{\text{max}} \cdot \frac{t+1}{\text{warmup}} & t < \text{warmup} \\ \text{lr}_{\text{min}} + \frac{\text{lr}_{\text{max}} - \text{lr}_{\text{min}}}{2} \left(1 + \cos\left(\pi \cdot \frac{t - \text{warmup}}{T - \text{warmup}}\right)\right) & t \geq \text{warmup} \end{cases}$$

Our hyperparameters: $\text{lr}_{\text{max}} = 6 \times 10^{-4}$, $\text{lr}_{\text{min}} = 6 \times 10^{-5}$, warmup = 200 steps.

6.3 Gradient Clipping

We clip the global gradient norm to 1.0:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

This prevents gradient explosions — rare but catastrophic events where a single bad batch produces enormous gradients that destabilize training.

6.4 Training Monitoring

During training we track:

- **Training loss**: averaged over gradient accumulation steps.
- **Learning rate**: confirms the schedule is applied correctly.
- **Tokens per second**: throughput metric.
- **MFU (Model FLOPs Utilization)**: what fraction of theoretical peak FLOPs the GPUs are achieving. 40–50% is typical for this model size.
- **Validation loss**: computed every `EVAL_INTERVAL` steps.

MFU is estimated as:

$$\text{MFU} = \frac{\text{tokens/sec} \times 6N}{P_{\text{peak}}}$$

where N is the parameter count and P_{peak} is the GPU's theoretical peak FLOPs in bf16.

6.5 Checkpointing

Checkpoints save:

- Model state dict (all parameters)
- Optimizer state dict (momentum, variance for every parameter)
- Current step and loss
- Model config (so we can reconstruct the model architecture)

We save at regular intervals (`SAVE_INTERVAL`) and whenever validation loss improves (`best.pt`).

6.6 The Complete Training Script

File: `microgpt/train.py`

```
"""MicroGPT training script (single-GPU and multi-GPU via DDP).

Usage:
    # Single GPU
    uv run python -m microgpt.train

    # Multi-GPU (8 GPUs on one node)
    uv run torchrun --nproc_per_node=8 -m microgpt.train

    # Multi-node (see vast.ai guide in Chapter 6)
```

```
    torchrun --nnodes=2 --nproc_per_node=8 --node_rank=$RANK \
        --master_addr=$MASTER --master_port=29500 -m microgpt.train
"""

import math
import os
import time

import torch
import torch.nn as nn
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

from .config import GPTConfig
from .model import MicroGPT
from .tokenizer import BPETokenizer
from .dataloader import create_dataloader
from .distributed import ddp_init, ddp_cleanup, is_master, print0


#
    ---------------------------------------------------------------------------
# Training hyperparameters
#
    ---------------------------------------------------------------------------

# Model
MODEL_CONFIG = GPTConfig()

# Optimization
MAX_STEPS = 20_000              # total training steps
WARMUP_STEPS = 200              # linear LR warmup
MAX_LR = 6e-4                   # peak learning rate
MIN_LR = 6e-5                   # final learning rate (10% of peak)
WEIGHT_DECAY = 0.1              # AdamW weight decay
BETA1, BETA2 = 0.9, 0.95       # AdamW betas
GRAD_CLIP = 1.0                 # max gradient norm

# Batch sizing
DEVICE_BATCH_SIZE = 16          # micro-batch per GPU per accumulation
    step
TOTAL_BATCH_TOKENS = 524_288    # ~0.5M tokens per optimizer step
# grad_accum_steps is computed from these at runtime

# Evaluation
EVAL_INTERVAL = 500             # steps between evaluations
EVAL_STEPS = 20                 # batches per evaluation

# Checkpointing
CHECKPOINT_DIR = "microgpt_checkpoints"
SAVE_INTERVAL = 2000            # steps between checkpoint saves

# Tokenizer
TOKENIZER_DIR = "tokenizer"
```

File: microgpt/train.py

```python
def get_lr(step: int) -> float:
    """Cosine learning rate schedule with linear warmup.

    1. Linear warmup from 0 to MAX_LR over WARMUP_STEPS.
    2. Cosine decay from MAX_LR to MIN_LR over remaining steps.
    """
    if step < WARMUP_STEPS:
        return MAX_LR * (step + 1) / WARMUP_STEPS
    if step >= MAX_STEPS:
        return MIN_LR
    # Cosine decay
    progress = (step - WARMUP_STEPS) / (MAX_STEPS - WARMUP_STEPS)
    return MIN_LR + 0.5 * (MAX_LR - MIN_LR) * (1.0 + math.cos(math.pi *
        progress))
```

File: microgpt/train.py

```python
def configure_optimizer(model: nn.Module) -> torch.optim.AdamW:
    """Configure AdamW with separate parameter groups for decay/no-decay.

    Weight decay is applied to all 2D+ parameters (weight matrices).
    Biases, norms, and embedding weights are not decayed.
    """
    decay_params = []
    no_decay_params = []

    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
        if param.ndim < 2:
            no_decay_params.append(param)
        else:
            decay_params.append(param)

    groups = [
        {"params": decay_params, "weight_decay": WEIGHT_DECAY},
        {"params": no_decay_params, "weight_decay": 0.0},
    ]

    optimizer = torch.optim.AdamW(
        groups,
        lr=MAX_LR,
        betas=(BETA1, BETA2),
        fused=True,  # faster fused CUDA kernel
    )
    return optimizer
```

6.7 Running on vast.ai

vast.ai is a GPU marketplace where you can rent 8×H100 nodes by the hour. Here is a minimal recipe:

1. **Select an instance** with 8×H100 (or A100) GPUs, Ubuntu, and PyTorch pre-installed.
2. **SSH in** and clone the repo:

```
git clone <your-repo-url> && cd learn-attn-docs
```

3. **Install deps**:

```
pip install uv && uv sync
```

4. **Download data**:

```
uv run python -m microgpt.dataset --num-shards 50
```

5. **Train a tokenizer** (or copy a pre-trained one):

```
# Extract text from first shard and train
uv run python -m microgpt.tokenizer \
    --input <text-file> --vocab-size 32768 --output tokenizer
```

6. **Launch training**:

```
uv run torchrun --nproc_per_node=8 -m microgpt.train
```

Tip: Multi-node training (e.g., 2 nodes of 8 GPUs each) requires setting `--nnodes`, `--node_rank`, `--master_addr`, and `--master_port` on each node. The `NCCL` backend handles inter-node communication over InfiniBand or Ethernet. For a GPT-2 scale model, a single 8-GPU node is sufficient.

6.8 Summary

The complete training recipe:

Component	Setting
Optimizer	AdamW (β_1=0.9, β_2=0.95, wd=0.1, fused)
LR schedule	Warmup 200 steps $\rightarrow$ cosine decay
Peak LR	6×10^{-4}
Effective batch	$\sim$0.5M tokens/step
Gradient clipping	Global norm 1.0
Precision	bf16 activations, fp32 master weights
Compilation	`torch.compile`
Checkpointing	Every 2000 steps + best val loss

Chapter 7

Efficient Inference

BabyGPT's `generate()` method recomputes the full attention matrix at every step. For a 1 024-context model with 12 layers, each generated token touches roughly 100 million multiply-adds worth of redundant key/value projections. In this chapter we implement a **KV cache** that stores previously computed keys and values, reducing per-token cost from $O(T)$ to $O(1)$ in the attention computation. We also cover nucleus (top-p) sampling.

7.1 The Autoregressive Bottleneck

Generating N tokens requires N forward passes. In each pass, self-attention computes keys and values for *all* positions in the sequence — including positions that were already processed.

Without a cache, generating token t requires:

- Computing Q, K, V for all t positions: $O(t \cdot d^2)$
- Attention matrix: $O(t^2 \cdot d)$

The K and V for positions 0 through $t-1$ are identical to what was computed when generating token $t-1$. Recomputing them is pure waste.

7.2 The KV Cache

The fix: cache K and V for all previous positions. When generating token t:

1. Compute Q, K, V for position t only ($O(d^2)$).
2. Append K and V to the cache.
3. Attend using Q for position t against *cached* K, V for all t positions ($O(t \cdot d)$).

Memory cost per layer:

$$2 \times T_{\max} \times n_{\text{kv_head}} \times d_{\text{head}} \times \text{dtype_size}$$

For our model (T=1024, 12 KV heads, d=64, bf16):

$$2 \times 1024 \times 12 \times 64 \times 2 = 3.1\,\text{MB per layer} \times 12 = 37.7\,\text{MB total}$$

With GQA (4 KV heads instead of 12), this drops to 12.6 MB — small enough to be irrelevant.

7.2.1 Implementation

File: `microgpt/generate.py`

```python
class KVCache:
    """Pre-allocated key/value cache for autoregressive generation.

    For each layer, stores keys and values of all previously processed
    tokens.  Shape per layer: (batch, max_seq_len, n_kv_head, head_dim).
    """

    def __init__(
        self,
        n_layers: int,
        max_seq_len: int,
        n_kv_head: int,
        head_dim: int,
        batch_size: int = 1,
        dtype: torch.dtype = torch.bfloat16,
        device: str = "cuda",
    ) -> None:
        self.n_layers = n_layers
        self.max_seq_len = max_seq_len
        self.pos = 0  # next write position

        # Pre-allocate: (n_layers, 2, batch, max_seq_len, n_kv_head,
            head_dim)
        # The '2' dimension is for K and V
        self.cache = torch.zeros(
            n_layers, 2, batch_size, max_seq_len, n_kv_head, head_dim,
            dtype=dtype, device=device,
        )

    def update(
        self,
        layer_idx: int,
        k: torch.Tensor,
        v: torch.Tensor,
    ) -> tuple[torch.Tensor, torch.Tensor]:
        """Write new K, V and return the full cached K, V.

        k, v: (B, T_new, n_kv_head, head_dim) --- new tokens
        Returns: (B, T_total, n_kv_head, head_dim) --- all cached tokens
        """
        T_new = k.size(1)
        self.cache[layer_idx, 0, :, self.pos : self.pos + T_new] = k
        self.cache[layer_idx, 1, :, self.pos : self.pos + T_new] = v
        # Return everything up to and including the new tokens
        end = self.pos + T_new
        return (
            self.cache[layer_idx, 0, :, :end],
            self.cache[layer_idx, 1, :, :end],
        )

    def advance(self, n: int) -> None:
        """Advance the write position by n tokens."""
        self.pos += n
```

7.3 Prefill and Decode Phases

Generation has two phases:

Prefill: process the entire prompt at once. This fills the KV cache for all prompt positions. Attention is causal over the prompt length.

Decode: generate one token at a time. Each step computes Q for the new position only and attends against the full cached K, V. Attention is not causal during decode (the single query can attend to all cached positions).

Tip: The prefill phase can be batched and parallelized just like training — it is compute-bound. The decode phase is memory-bound because each step only processes one token but reads the full KV cache. This is why KV cache size matters for inference throughput.

7.4 Top-p (Nucleus) Sampling

In addition to temperature and top-k (covered in the BabyGPT tutorial), we implement **nucleus sampling** (Holtzman et al., 2020):

1. Sort tokens by probability (descending).
2. Compute the cumulative probability.
3. Keep the smallest set of tokens whose cumulative probability $\geq p$.
4. Re-normalize and sample.

Top-p is adaptive: when the model is confident, it keeps few tokens; when uncertain, it keeps many. Typical values: $p = 0.9$ or $p = 0.95$.

```
if top_p is not None:
    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    cumprobs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)
    remove_mask = cumprobs - F.softmax(sorted_logits, dim=-1) >= top_p
    sorted_logits[remove_mask] = float("-inf")
    logits = sorted_logits.scatter(1, sorted_indices, sorted_logits)
```

7.5 Summary

Efficient inference requires:

1. **KV cache**: store K, V for previously processed tokens; compute only Q for new tokens.
2. **Prefill + decode**: batch-process the prompt, then generate one token at a time.
3. **Sampling strategies**: temperature, top-k, and top-p for controlling output diversity.

The full generation script is in `microgpt/generate.py`, including a CLI for generating text from a trained checkpoint.

Chapter 8

Evaluation

How do you know if your language model is any good? In this chapter we cover **perplexity** and **bits per byte** (a tokenizer-independent metric), implement validation-loss tracking, and briefly survey zero-shot benchmarks and scaling laws.

8.1 Perplexity

Perplexity is the exponential of the average cross-entropy loss:

$$\mathrm{PPL} = \exp\left(\frac{1}{N}\sum_{i=1}^{N} -\log p(x_i \mid x_{<i})\right) = e^{\mathcal{L}}$$

Intuitively, perplexity measures how "surprised" the model is by the text. A perplexity of 10 means the model is as uncertain as if it were choosing uniformly among 10 equally likely tokens at each position.

Lower is better. A random model over a 32K vocabulary has perplexity ∼32 768. GPT-2 124M typically achieves perplexity ∼25–30 on web text.

```
def perplexity(loss: float) -> float:
    import math
    return math.exp(loss)
```

8.2 Bits Per Byte (BPB)

Perplexity depends on the tokenizer: a model with a 50K vocabulary and a model with a 32K vocabulary are not directly comparable by perplexity because they predict over different token sets.

Bits per byte normalizes by the number of raw bytes in the text, making it tokenizer-independent:

$$\mathrm{BPB} = \frac{\sum_i \mathcal{L}(x_i)}{\text{total bytes}} \cdot \frac{1}{\ln 2}$$

Each token contributes its cross-entropy loss, weighted by how many UTF-8 bytes it represents. The $1/\ln 2$ converts from nats (natural log) to bits.

BPB values for English text typically range from 0.8 (strong model) to 1.5 (weak model). Random: ∼8 bits/byte.

File: microgpt/eval.py

```
@torch.no_grad()
def compute_bpb(
    model: MicroGPT,
    val_loader,
    tokenizer: BPETokenizer,
    num_batches: int = 50,
) -> float:
    """Compute bits per byte (BPB) on validation data.

    BPB is tokenizer-independent: it measures compression in terms of
    raw bytes rather than tokens.  This allows fair comparison between
    models with different tokenizers.

    BPB = mean_cross_entropy / ln(2) * (tokens / bytes)
    """
    model.eval()
    total_loss = 0.0
    total_tokens = 0
    total_bytes = 0

    for i, (inputs, targets) in enumerate(val_loader):
        if i >= num_batches:
            break

        logits, _ = model(inputs)
        # Per-token cross-entropy
        B, T, V = logits.shape
        loss_per_token = F.cross_entropy(
            logits.reshape(-1, V),
            targets.reshape(-1),
            reduction="none",
        ).view(B, T)

        # Count bytes per token
        for b in range(B):
            for t in range(T):
                token_id = targets[b, t].item()
                try:
                    token_bytes = tokenizer.token_to_bytes(token_id)
                    n_bytes = len(token_bytes)
                except Exception:
                    n_bytes = 1  # fallback for special tokens

                total_loss += loss_per_token[b, t].item()
                total_bytes += n_bytes
                total_tokens += 1

    # BPB = total_nats / total_bytes / ln(2)
    import math
    bpb = total_loss / max(total_bytes, 1) / math.log(2)
    return bpb
```

8.3 Validation Loss

During training, we compute validation loss every `EVAL_INTERVAL` steps on a held-out shard (the last parquet file). This is the most important metric for detecting:

- **Overfitting**: val loss starts increasing while train loss continues decreasing. Unlikely at this data scale but important to monitor.
- **Training instability**: sudden spikes in val loss indicate exploding gradients or bad learning rate.
- **Convergence**: val loss plateaus, indicating diminishing returns from further training.

8.4 Zero-Shot Benchmarks

The GPT-2 paper's key claim was that language models can perform tasks zero-shot. Standard benchmarks include:

Benchmark	What it tests
HellaSwag	Commonsense reasoning: pick the most plausible continuation of a scenario.
LAMBADA	Long-range prediction: predict the last word of a passage.
ARC (AI2 Reasoning Challenge)	Science questions from grade-school exams.
WinoGrande	Pronoun resolution requiring commonsense.

For zero-shot evaluation, the model is given each candidate answer and the one with the lowest perplexity (or highest log-probability) is selected. No fine-tuning is performed.

A well-trained GPT-2 124M typically achieves:

- HellaSwag: ∼30–35% accuracy (random baseline: 25%)
- LAMBADA: ∼40–50% accuracy

8.5 Scaling Laws

Kaplan et al. (2020) and Hoffmann et al. (2022, "Chinchilla") established that language model loss follows a power law in compute:

$$L(C) \approx L_0 + A \cdot C^{-\alpha}$$

Key findings:

- Loss decreases predictably with more compute (larger models, more data, longer training).
- **Compute-optimal training**: for a fixed compute budget, there is an optimal ratio of model size to data size. Chinchilla's ratio: ∼20 tokens per parameter.
- For our 135M parameter model, compute-optimal training would use ∼2.7B tokens. Training on 10B tokens is "over-training" by Chinchilla standards, but smaller models benefit from seeing more data.

Tip: Nanochat reports a CORE score (Composite Overall Reasoning Evaluation) that aggregates multiple benchmarks. Their GPT-2 reproduction achieves a CORE score of ∼0.257, matching the original GPT-2 124M. This provides a useful single-number target for verifying your training run.

8.6 Summary

Evaluating a language model:

1. **Validation loss**: primary training metric; detect overfitting and instability.
2. **Perplexity**: e^{loss}; intuitive but tokenizer-dependent.
3. **Bits per byte**: tokenizer-independent compression metric; enables cross-model comparison.
4. **Zero-shot benchmarks**: HellaSwag, LAMBADA, ARC test real capabilities.
5. **Scaling laws**: predict performance at larger scale; guide compute allocation.

The evaluation script is in `microgpt/eval.py`, runnable as:

Shell — run in your terminal

```shell
uv run python -m microgpt.eval \
    --checkpoint microgpt_checkpoints/best.pt \
    --tokenizer tokenizer
```

Appendix A

Hardware Requirements

This appendix provides concrete hardware requirements and recommendations for training and running MicroGPT. All estimates assume the default configuration: 135M parameters, 1 024 context length, 32K vocabulary, bf16 mixed precision.

A.1 Memory Breakdown

The GPU memory footprint has four components:

Component	Size	Notes
Model weights (fp32 master)	0.54 GB	135M $\times$ 4 bytes
AdamW optimizer states (m, v)	1.08 GB	2 $\times$ 135M $\times$ 4 bytes
Gradients (fp32)	0.54 GB	135M $\times$ 4 bytes
Activations (bf16, batch=16)	~6 GB	Scales with batch size
Total (training, batch=16)	**~8.2 GB**	

Activations dominate and scale linearly with batch size. Reducing `DEVICE_BATCH_SIZE` from 16 to 8 cuts activation memory roughly in half; gradient accumulation compensates to maintain the same effective batch size.

For **inference only**, the footprint is much smaller — the model in bf16 plus the KV cache:

Component	Size	Notes
Model weights (bf16)	0.27 GB	135M $\times$ 2 bytes
KV cache (12 layers, bf16)	0.04 GB	12 $\times$ 2 $\times$ 1024 $\times$ 12 $\times$ 64 $\times$ 2 bytes
Total (inference)	**~0.3 GB**	

A.2 Dataset Storage

FineWeb-Edu shards are compressed Parquet files hosted on HuggingFace:

Shards	Disk	~Tokens	Use case
2	~150 MB	~200M	Smoke test / debugging
10 (default)	~700 MB	~1B	Quick experiment
50	~3.5 GB	~5B	Solid training run
99 (all)	~7 GB	~10B	Full reproduction

The last shard is reserved for validation. For the default 20K-step run at 0.5M tokens/step (∼10B tokens total), download all 99 shards. With fewer shards the dataloader loops (multi-epoch), which still works but may overfit.

Checkpoint storage is modest: each checkpoint is ∼1.1 GB (model + optimizer state in fp32).

A.3 Training: GPU Recommendations

Training time depends on GPU throughput. The estimates below assume 40% Model FLOPs Utilization (MFU), which is realistic for this model size with `torch.compile` and bf16:

GPU	VRAM	bf16 TFLOPS	1-GPU time	8-GPU time
RTX 3080	10 GB	30	∼200 h	∼25 h
RTX 3090	24 GB	36	∼166 h	∼21 h
RTX 4090	24 GB	83	∼72 h	∼9 h
A100 (40/80 GB)	40–80 GB	312	∼19 h	∼2.4 h
H100 SXM	80 GB	990	∼6 h	∼45 min

A.3.1 Minimum

A **single GPU with 10 GB VRAM** (e.g. RTX 3080) can train MicroGPT. Reduce `DEVICE_BATCH_SIZE` to 4–8 to fit in memory; gradient accumulation will maintain the effective batch size. Training will take roughly a week.

A.3.2 Recommended

- **Budget option**: 1× RTX 4090 on vast.ai (∼$0.40/hr). Finishes in ∼3 days, total cost ∼$35.
- **Sweet spot**: 1× A100 80 GB (∼$1.50/hr). Comfortable VRAM headroom, finishes in under a day, ∼$30.
- **Fast iteration**: 8× H100 node (∼$25/hr on vast.ai). Under an hour for a full run, ∼$25. Matches nanochat's reported training time for GPT-2 reproduction.

A.3.3 Batch size adjustments

If you hit out-of-memory errors, reduce `DEVICE_BATCH_SIZE` in `microgpt/train.py`. The gradient accumulation logic automatically increases `grad_accum_steps` to compensate:

$$A = \left\lfloor \frac{B_{\text{total_tokens}}}{B_{\text{device}} \times T \times W} \right\rfloor$$

`DEVICE_BATCH_SIZE`	Peak VRAM	Accum steps (1 GPU)	Notes
16	∼8.2 GB	32	Default
8	∼5.1 GB	64	Fits on 8 GB GPUs
4	∼3.6 GB	128	Fits on 6 GB GPUs

Smaller micro-batches increase the number of accumulation steps per optimizer step, which is slightly slower (more kernel launches) but mathematically equivalent.

A.4 Inference and Evaluation

Inference (text generation) and evaluation (computing validation loss / BPB) have much lower requirements than training:

Task	Min VRAM	Notes
Text generation (batch=1)	∼0.5 GB	Model (bf16) + KV cache
Evaluation (`microgpt.eval`, batch=8)	∼3 GB	No optimizer states

Any modern GPU with 2+ GB VRAM can run inference. This includes laptop GPUs (e.g. RTX 3050 Mobile with 4 GB). CPU-only inference also works but is ∼50× slower.

For evaluation, a single GPU with 4+ GB VRAM is sufficient. Reduce `--batch-size` if needed.

Tip: If you trained on a multi-GPU cluster but want to evaluate on a single consumer GPU, just load the checkpoint — the model weights are the same regardless of how many GPUs were used for training. DDP only affects gradient averaging, not the final model.

A.5 Summary

Task	Minimum	Recommended
Training	1× GPU, 10 GB VRAM	1× A100 80 GB or 8× H100
Inference	Any GPU, 2 GB VRAM	Any GPU, 4+ GB VRAM
Evaluation	1× GPU, 4 GB VRAM	1× GPU, 8+ GB VRAM
Disk (dataset)	150 MB (2 shards)	7 GB (all 99 shards)
Disk (checkpoints)	1.1 GB	5+ GB (multiple saves)